

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Michał Kozłowski

Webowy dziennik ocen studenta

Web-based student gradebook

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr hab. Ireneusz Morawski

Wrocław, 4 marca 2026

Spis treści

1	Wstęp	8
1.1	Kontekst i motywacja	8
1.2	Funkcjonalności USOS w opisywanym zakresie	8
1.3	Cel i zakres pracy	8
1.4	Struktura pracy	9
2	Założenia i wymagania projektowe	10
2.1	Założenia projektu	10
2.2	Wymagania funkcjonalne	11
2.3	Wymagania niefunkcjonalne	12
3	Zgodność z rozporządzeniem RODO	13
3.1	Zakres przetwarzanych danych osobowych	13
3.2	Minimalizacja przechowywanych danych osobowych	13
3.3	Podstawa prawna przetwarzania danych	13
3.4	Kontrola dostępu a zasada ograniczonego dostępu	14
3.5	Wymagania techniczne w zakresie bezpieczeństwa danych	14
3.6	Podsumowanie analizy zgodności z RODO	15
4	Architektura systemu	16
4.1	Porównanie wzorców architektonicznych na poziomie systemu	16
4.2	Kontekst systemu	17
4.3	Uzasadnienie wyboru architektury mikroserwisowej	18
4.4	Komponenty systemu	18
4.5	Role pełnione przez poszczególne komponenty	20
4.5.1	Frontend	20
4.5.2	API Gateway	20
4.5.3	Serwis autoryzacji	20
4.5.4	Serwis grup	20
4.5.5	Serwisy ocen i obecności	20
4.5.6	Serwis podsumowania	20
4.5.7	Serwis sprawdzianów	21
4.6	Izolacja danych w mikroserwisach	21
4.7	Przyjęte wzorce na poziomie serwisu	21
5	Przegląd technologii wykorzystanych do budowy aplikacji	22
5.1	Spring Boot	22
5.1.1	Charakterystyka	22
5.1.2	Zastosowanie w projekcie	22
5.1.3	Uzasadnienie wyboru	22
5.2	React	23
5.2.1	Charakterystyka	23
5.2.2	Zastosowanie w projekcie	23
5.2.3	Uzasadnienie wyboru	23
5.3	MySQL	23
5.3.1	Charakterystyka	23

5.3.2	Zastosowanie w projekcie	23
5.3.3	Uzasadnienie wyboru	24
5.4	Redis	24
5.4.1	Charakterystyka	24
5.4.2	Zastosowanie w projekcie	24
5.4.3	Uzasadnienie wyboru	24
5.5	Azure Computer Vision	24
5.5.1	Charakterystyka	24
5.5.2	Zastosowanie w projekcie	24
5.5.3	Uzasadnienie wyboru	25
5.6	Docker	25
5.6.1	Charakterystyka	25
5.6.2	Zastosowanie w projekcie	25
5.6.3	Uzasadnienie wyboru	25
6	Implementacja backendu	26
6.1	Współdzielone zależności	26
6.2	Bezpieczeństwo	26
6.2.1	JWT	26
6.2.2	Role i dostęp	27
6.2.3	Implementacja	27
6.3	Komunikacja między mikroserwisami	27
6.4	Serwis autoryzacji	27
6.4.1	Diagram komponentów	27
6.4.2	Proces autoryzacji wewnątrz systemu	29
6.4.3	JWKS	29
6.4.4	Proces Autoryzacji z USOS API	29
6.4.5	Schemat bazy danych	30
6.4.6	Komunikacja z USOS API	31
6.5	Serwis grup	31
6.5.1	Diagram komponentów	31
6.5.2	Pobieranie danych	33
6.5.3	Grupy zajęciowe	33
6.5.4	Aktualny semestr	33
6.5.5	Schemat bazy danych	33
6.6	API Gateway	35
6.6.1	Routing	35
6.6.2	Filtry	35
6.6.3	Odporność na błędy	35
6.7	Serwis obecności	36
6.7.1	Diagram komponentów	36
6.7.2	Zarządzanie listami obecności	36
6.7.3	Przeglądanie obecności	37
6.7.4	Aktualny semestr	37
6.7.5	Schemat bazy danych	37
6.8	Serwis ocen	38
6.8.1	Diagram komponentów	38

6.8.2	Zarządzanie ocenami	39
6.8.3	Przeglądanie ocen	39
6.8.4	Schemat bazy danych	39
6.9	Serwis podsumowania	40
6.9.1	Diagram komponentów	40
6.9.2	Funkcjonalność	40
6.10	Serwis sprawdzianów	41
6.10.1	Diagram komponentów	41
6.10.2	Zarządzanie szablonami sprawdzianów	41
6.10.3	Schemat bazy danych	42
6.10.4	Przebieg procesu automatycznego oceniania sprawdzianów	42
6.10.5	Specyfika wykorzystanego modelu OCR	42
6.10.6	Format punktacji	43
6.10.7	Generacja tabeli punktacji	43
6.10.8	Wstępne przetwarzanie obrazu	44
6.10.9	Analiza tabeli punktacji	44
6.10.10	Działanie algorytmu analizy tabeli punktacji	44
6.10.11	Analiza złożoności algorytmu	47
7	Implementacja frontendu	49
7.1	Architektura	49
7.1.1	Podejście modułowe	49
7.1.2	Funkcje typu hook jako warstwa logiki	49
7.2	System uwierzytelniania i kontroli dostępu	49
7.2.1	Proces autoryzacji przez USOS	49
7.2.2	Zarządzanie sesją	50
7.2.3	Kontrola dostępu oparta na rolach	50
7.3	Moduły funkcjonalne	50
7.3.1	Moduł kursów	50
7.3.2	Moduł sprawdzianów	51
7.4	Komunikacja z backendem	51
8	Testowanie	52
8.1	Testy jednostkowe	52
8.2	Testy manualne	52
8.3	Testowanie systemu analizy zdjęć tabeli punktacji	52
9	Wdrożenie systemu	53
9.1	Środowisko wdrożeniowe	53
9.2	Architektura kontenerów	53
9.3	Sieć i izolacja	53
9.4	Szyfrowanie komunikacji	54
9.4.1	TLS na warstwie HTTP	54
9.4.2	SSL w połączeniach z bazami danych	54
9.5	Trwałość danych	54
9.6	Kolejność uruchamiania i kontrole stanu	54
9.7	Zarządzanie sekretami	54
9.8	Przepływ żądania	55

10 Podsumowanie	56
10.1 Napotkane problemy	56
10.1.1 Architektura mikroserwisowa	56
10.1.2 Integracja z USOS API	56
10.1.3 Moduł automatycznego oceniania sprawdzianów	56
10.2 Wnioski końcowe	56
10.3 Możliwości rozwoju	57
Bibliografia	58

Streszczenie

Niniejsza praca inżynierska przedstawia projekt i implementację webowego dziennika ocen studenta – systemu wspomagającego zarządzanie dydaktyką na uczelniach wyższych. Motywacją do stworzenia projektu były ograniczenia systemu USOS w zakresie zarządzania ocenami częściowymi i frekwencją studentów w trakcie semestru. Backend został zrealizowany w architekturze mikroservisowej z wykorzystaniem frameworka Spring Boot, a frontend jako Single Page Application z wykorzystaniem biblioteki React. System integruje się z USOS za pośrednictwem protokołu OAuth 1.0a. Wyróżniającą funkcjonalnością jest moduł automatycznego oceniania sprawdzianów wykorzystujący usługę OCR Azure Computer Vision. Całość została wdrożona w środowisku kontenerowym Docker.

Słowa kluczowe: dziennik ocen, Spring Boot, React, Docker, OCR, JWT, USOS

Abstract

This engineering thesis presents the design and implementation of a web-based student gradebook – a system supporting academic management at higher education institutions. The project was motivated by the limitations of the USOS system regarding the management of partial grades and student attendance during the semester. The backend was implemented using a microservice architecture with the Spring Boot framework, while the frontend was developed as a Single Page Application using the React library. The system integrates with USOS via the OAuth 1.0a protocol. A distinguishing feature is the automatic exam grading module utilizing the Azure Computer Vision OCR service. The entire system was deployed in a Docker container environment.

Keywords: gradebook, Spring Boot, React, Docker, OCR, JWT, USOS

1 Wstęp

1.1 Kontekst i motywacja

Zarządzanie dydaktyką na uczelniach wyższych wiąże się z koniecznością prowadzenia rozbudowanej dokumentacji. W jej skład wchodzi między innymi oceny i obecności. Podstawowym narzędziem obsługi toku studiów jest USOS¹, który pełni jednak przede wszystkim rolę centralnego rejestru danych akademickich, nie oferując wykładowcom intuicyjnych narzędzi do bieżącego zarządzania ocenami cząstkowymi i obecnością w trakcie semestru.

Motywacją do powstania niniejszej pracy były obserwacje poczynione podczas studiów. Studenci rzadko mają bezpośredni dostęp do bieżących ocen cząstkowych i stanu swojej frekwencji. Wykładowcy zazwyczaj wprowadzają do systemu USOS jedynie oceny końcowe. Informowanie studentów o wynikach cząstkowych i obecnościach odbywa się najczęściej drogą mailową lub ustnie. Promotor niniejszej pracy, prowadzący przedmiot z cotygodniowymi kolokwiami, wskazał na szczególną uciążliwość tego procesu. Każdorazowe informowanie kilkudziesięciu studentów o wynikach jest czasochłonne i podatne na błędy.

1.2 Funkcjonalności USOS w opisywanym zakresie

Moduł ocen cząstkowych istnieje w systemie USOS w postaci modułu Sprawdziany. Jest on jednak opcjonalny – nie wszystkie uczelnie go wdrażają, a jego konfiguracja wymaga wsparcia administracji systemu. W praktyce prowadzący często nie mają do niego dostępu.

Dużą słabością systemu USOS jest brak modułu do elektronicznego sprawdzania obecności. Prowadzący może jedynie pobrać listę studentów w formacie PDF lub CSV i zaznaczyć obecność poza systemem. Część uczelni rozwiązuje ten problem przez zewnętrzne systemy.

System USOS nie oferuje także zintegrowanego widoku podsumowania wyników pracy studenta w grupie zajęciowej. Brakuje miejsca, gdzie prowadzący mógłby zobaczyć zestawienie ocen cząstkowych, aktywności i frekwencji. Taka funkcjonalność na pewno pomogłaby w wystawianiu ocen końcowych.

1.3 Cel i zakres pracy

Celem pracy jest kompensacja opisywanych ograniczeń systemu USOS oraz ułatwienie pracy wykładowcom za pośrednictwem zintegrowanego narzędzia w formie aplikacji webowej.

Zakres pracy obejmuje projekt architektury systemu, implementacje backendu² oraz frontendu³, integrację z systemami zewnętrznymi oraz wdrożenie aplikacji. W niniejszej pracy położono nacisk na praktyczną realizację systemu.

¹USOS (Uniwersytecki System Obsługi Studiów) – system informatyczny zarządzający tokiem studiów, powszechnie stosowany na polskich uczelniach.

²backend – warstwa logiki serwerowej aplikacji.

³frontend – warstwa kliencka, interfejs użytkownika.

1.4 Struktura pracy

Pracę podzielono na 10 rozdziałów, gdzie kolejno poruszane są kwestie: założeń i wymagań projektowych, zgodności z rozporządzeniem RODO[1], architektury systemu, wykorzystanych technologii, implementacji backendu, implementacji frontendu, testowania aplikacji oraz wdrożenia. Ostatnim rozdziałem jest podsumowanie, w którym zawarte są wnioski końcowe, opis napotkanych problemów, a także pomysły na dalszy rozwój projektu.

2 Założenia i wymagania projektowe

2.1 Założenia projektu

Celem projektu jest stworzenie intuicyjnej aplikacji webowej, która ułatwi pracę wykładowców i usprawni dostęp studentów do informacji o swoich ocenach i nieobecnościach. Aplikacja powinna dostarczać wykładowcom takie funkcjonalności jak automatyczne pobranie grup zajęciowych, zarządzanie ocenami, zarządzanie listami obecności oraz automatyczne ocenianie sprawdzianów, poprzez analizę zdjęć wypełnionych tabel z punktacją. Powinna także pomagać w wystawieniu oceny końcowej poprzez tworzenie podsumowań pracy na zajęciach każdego ze studentów w grupie. Student korzystając z aplikacji powinien móc szybko i w sposób przejrzysty zobaczyć wyniki swojej dotychczasowej pracy w obrębie poszczególnych kursów akademickich.

Aplikacja jest przeznaczona dla uczelni wyższych korzystających z systemu USOS. Nie ma na celu jego zastąpienia, stanowi narzędzie dopełniające, które wymaga aktywnego konta w tym systemie.

Elementem wyróżniającym zakładany projekt jest funkcjonalność automatycznego oceniania sprawdzianów. Wykładowca tworzy szablon określając tytuł, liczbę zadań oraz progi procentowe dla poszczególnych ocen. Następnie pobiera wygenerowaną na podstawie tego szablonu tabelkę, którą dołącza do swojego sprawdzianu przed drukiem. Studenci pisząc sprawdzian wprowadzają do tabelki swój numer indeksu. Wykładowca podczas sprawdzania wpisuje punktację, a następnie przesyła zdjęcia wypełnionej tabelki do systemu. Aplikacja wykorzystuje technologię OCR⁴ do odczytu pisma, po czym za pomocą algorytmu odnajduje numer indeksu oraz paruje punktację z odpowiednimi zadaniami. Po zakończonej analizie wykładowca zatwierdza wynik, a ocena zostaje wystawiona.

Założono wykorzystanie następujących technologii:

- Spring Boot
- React.js
- MySQL
- Redis
- Azure Computer Vision
- Docker

⁴OCR (*ang. Optical Character Recognition*, pol. Optyczne rozpoznawanie znaków) – technologia umożliwiająca automatyczne rozpoznawanie tekstu ze zdjęć i skanów.

2.2 Wymagania funkcjonalne

Poniżej przedstawiono listę wymagań funkcjonalnych systemu z podziałem na role użytkowników.

W przypadku wykładowców system umożliwia:

- zalogowanie się do systemu za pośrednictwem systemu USOS,
- pobranie i synchronizację grup zajęciowych z systemu USOS,
- zarządzanie ocenami studentów w grupach zajęciowych,
- zarządzanie listami obecności w grupach zajęciowych,
- generowanie podsumowań ocen, aktywności i frekwencji studentów w grupach zajęciowych,
- tworzenie szablonów sprawdzianów (tytuł, liczba zadań, progi procentowe),
- wygenerowanie i pobranie tabeli punktacji wygenerowanej na bazie szablonu,
- przysyłanie zdjęć wypełnionych tabeli do systemu,
- rozpoznawanie numeru indeksu i punktacji w tabelkach ze zdjęć,
- wystawianie ocen studentom ze sprawdzianów na podstawie wyników analizy zdjęć tabeli;

W przypadku studentów system umożliwia:

- zalogowanie się do systemu za pośrednictwem systemu USOS,
- przeglądanie swoich ocen zdobytych w poszczególnych kursach,
- przeglądanie stanu swojej frekwencji na zajęciach w poszczególnych kursach,
- generowanie swoich podsumowań ocen, aktywności i frekwencji w poszczególnych kursach;

2.3 Wymagania niefunkcjonalne

Poniżej przedstawiono wymagania niefunkcjonalne systemu.

- trwałe przechowywanie danych w relacyjnej bazie danych,
- wykorzystanie klucza do API⁵ USOS UWR,
- komunikacja z USOS UWR poprzez endpointy⁶ REST⁷ API,
- zabezpieczenie systemu przed nieautoryzowanym dostępem,
- przetwarzanie danych osobowych zgodnie z RODO,
- integracja z modelem odczytywania tekstu ze zdjęć,
- implementacja algorytmu oceniającego sprawdziany na podstawie szablonu i odpowiedzi modelu,
- skalowalność i łatwość rozbudowy systemu,
- obserwowalność działania systemu;

⁵API (ang. *Application Programming Interface*, pol. Interfejs programowania aplikacji) – zestaw reguł i protokołów umożliwiający komunikację między różnymi komponentami oprogramowania.

⁶endpoint (pol. punkt końcowy) – adres URL identyfikujący konkretny zasób lub operację udostępnianą przez interfejs API, pod który kierowane są żądania sieciowe.

⁷REST (ang. *Representational State Transfer*, pol. Transfer stanu za pomocą reprezentacji zasobów) – styl architektoniczny oprogramowania definiujący zasady komunikacji między systemami za pośrednictwem protokołu HTTP, oparty na operacjach na zasobach identyfikowanych przez adresy URL.

3 Zgodność z rozporządzeniem RODO

Projektowanie i wdrażanie systemów informatycznych przetwarzających dane osobowe nakłada na twórcę obowiązek uwzględnienia przepisów wynikających z Rozporządzenia Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 roku w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych i w sprawie swobodnego przepływu takich danych, powszechnie określanego skrótem RODO. Niniejszy rozdział zawiera analizę zaprojektowanej aplikacji webowej dziennika ocen pod względem zgodności z wymogami tego rozporządzenia.

3.1 Zakres przetwarzanych danych osobowych

Aplikacja przetwarza następujące dane osobowe studentów: imię, skrót nazwiska (trzy pierwsze litery) oraz numer indeksu. Zgodnie z art. 4 pkt 1 RODO danymi osobowymi są wszelkie informacje dotyczące zidentyfikowanej lub możliwej do zidentyfikowania osoby publicznej, przy czym jako przykład identyfikatora rozporządzenie wskazuje wprost numer identyfikacyjny. Numer indeksu spełnia tę definicję, jest unikalnym identyfikatorem przypisanym do konkretnego studenta. Stanowi dane osobowe w rozumieniu RODO, choć nie jest zaliczany do kategorii danych wrażliwych wymienionych w art. 9 rozporządzenia.

3.2 Minimalizacja przechowywanych danych osobowych

Zastosowanie skróconej formy nazwiska zamiast jego pełnego brzmienia jest przejawem zasady minimalizacji danych wyrażonej w art. 5 ust. 1 lit. c RODO. Zasada ta stanowi, że przetwarzane dane powinny być adekwatne, stosowne oraz ograniczone do tego, co niezbędne do celów, w których są przetwarzane. W kontekście dziennika ocen skrót nazwiska w połączeniu z imieniem i numerem indeksu jest wystarczający do jednoznacznej identyfikacji studenta w ramach systemu, a jednocześnie ogranicza ekspozycje pełnych danych osobowych.

3.3 Podstawa prawna przetwarzania danych

Art. 6 RODO wymaga, aby każda operacja przetwarzania danych osobowych opierała się na co najmniej jednej z wymienionych w nim przesłanek legalizujących. W przypadku systemu informatycznego prowadzonego przez uczelnię wyższą właściwą podstawą prawną jest art. 6 ust. 1 lit. c RODO, tj. niezbędność przetwarzania do wypełnienia obowiązku prawnego ciążącego na administratorze. Obowiązek ten wynika bezpośrednio z ustawy z dnia 20 lipca 2018 roku – Prawo o szkolnictwie wyższym i nauce[2], która nakłada na uczelnię obowiązek prowadzenia dokumentacji przebiegu studiów, w tym ewidencji ocen i frekwencji studentów.

Powyższe oznacza, że uczelnia jako administrator danych nie jest zobowiązana do pozyskiwania odrębnej zgody studenta na przetwarzanie jego danych w omawianym systemie. Przetwarzanie danych w celu obsługi toku studiów jest nieodłącznym elementem realizacji stosunku prawnego łączącego studenta z uczelnią. Istotne jest jednak, aby uczelnia dopełniła obowiązku informacyjnego wynikającego z art. 13 RODO,

przekazując studentowi przy immatrykulacji lub w innym stosownym momencie informacje o tożsamości administratora, celach i podstawach przetwarzania danych oraz przysługujących mu prawach.

3.4 Kontrola dostępu a zasada ograniczonego dostępu

Kluczowym aspektem zgodności z RODO jest zastosowanie modelu kontroli dostępu opartego na rolach. W zaprojektowanym systemie wyróżniono dwie role użytkowników: studenta i wykładowcę. Student ma dostęp wyłącznie do swoich ocen i danych dotyczących obecności, natomiast wykładowca może przeglądać jedynie dane studentów przypisanych do prowadzonych przez niego grup zajęciowych.

Opisane powyżej podejście realizuje zasadę integralności i poufności wyrażoną w art. 5 ust. 1 lit. f RODO, nakazującą przetwarzanie danych osobowych w sposób zapewniający odpowiednie bezpieczeństwo, w tym ochronę przed nieuprawnionym dostępem. Jednocześnie wpisuje się w zasadę ograniczenia celu opisaną w art. 5 ust. 1 lit. b RODO – dane są dostępne wyłącznie w zakresie niezbędnym do realizacji konkretnego celu dydaktycznego. Ograniczenie dostępu musi być egzekwowane na poziomie backendu, a nie wyłącznie przez frontend, gdyż tylko wtedy stanowi skuteczne zabezpieczenie techniczne w rozumieniu art. 32 RODO.

3.5 Wymagania techniczne w zakresie bezpieczeństwa danych

Art. 32 RODO zobowiązuje administratora do wdrożenia odpowiednich środków technicznych i organizacyjnych zapewniających stopień bezpieczeństwa odpowiadający ryzyku przetwarzania. W odniesieniu do omawianej aplikacji do najważniejszych środków technicznych należą:

- szyfrowanie komunikacji z wykorzystaniem protokołu TLS⁸ (HTTPS), zapobiegające przechwyceniu danych przesyłanych między klientem a serwerem,
- bezpieczne przechowywanie tokenów dostępu z pomocą szyfrowania oraz ograniczenie do minimum czasu ich przechowywania, co uniemożliwia ich odczyt w przypadku wycieku bazy danych,
- uwierzytelnianie użytkowników za pomocą tokenów JWT⁹ z określonym czasem wygaśnięcia,
- rejestrowanie dostępu do danych osobowych (logowanie zdarzeń) w celu umożliwienia wykrycia oraz udokumentowania ewentualnych naruszeń;

Ponadto art. 25 RODO wprowadza zasadę ochrony danych w fazie projektowania oraz ochrony danych jako ustawienia domyślnego. Oznacza to, że już na etapie projektowania architektury systemu należy uwzględnić mechanizmy ochrony danych osobowych,

⁸TLS (ang. Transport Layer Security) – kryptograficzny protokół komunikacyjny zapewniający szyfrowanie danych przesyłanych w sieci.

⁹JWT (ang. JSON Web Token) – otwarty standard definiujący sposób bezpiecznego przesyłania informacji między stronami w postaci obiektu JSON, podpisanego cyfrowo.

a domyślne ustawienia aplikacji powinny zapewniać jak najwyższy poziom prywatności – co w omawianym projekcie realizowane jest przez restrykcyjny model dostępu do danych.

3.6 Podsumowanie analizy zgodności z RODO

Przeprowadzona analiza wskazuje, że zaprojektowana aplikacja webowa dziennika ocen może być użytkowana w sposób zgodny z rozporządzeniem RODO. Zakres przetwarzania danych osobowych ograniczono do niezbędnego minimum zgodnie z zasadą minimalizacji danych. Podstawą prawną przetwarzania danych jest obowiązek prawny ciążyący na uczelni wynikający z przepisów ustawy o szkolnictwie wyższym i nauce. Zaimplementowany model kontroli dostępu opartego na rolach zapewnia, że każdy użytkownik ma dostęp wyłącznie do danych niezbędnych do realizacji jego zadań. Wdrożenie opisanych środków technicznych gwarantuje zaś bezpieczeństwo danych na poziomie adekwatnym do ryzyka związanego z przetwarzaniem danych w systemie edukacyjnym.

4 Architektura systemu

Odpowiednie zaplanowanie architektury systemu jest kluczowym etapem procesu wytwarzania oprogramowania, wpływającym na jego skalowalność, niezawodność, łatwość utrzymania oraz możliwości rozwoju. W niniejszym rozdziale porównano różne wzorce architektoniczne oraz uzasadniono poczynione wybory. Architektura systemu została przedstawiona w sposób inspirowany modelem C4[3], opisując system na poziomie kontekstu oraz komponentów systemu.

4.1 Porównanie wzorców architektonicznych na poziomie systemu

Dwoma najpopularniejszymi wzorcami architektonicznymi na poziomie systemu są: architektura monolityczna oraz architektura mikroserwisowa. Wybór między nimi jest jedną z najważniejszych decyzji projektowych, która determinuje sposób organizacji kodu oraz strategię wdrożenia.

Architektura monolityczna charakteryzuje się traktowaniem systemu jako jednolitej całości – cały backend działa jako jeden proces, współdzieląc bazę danych. Możliwy jest przy tym podział na warstwę logiczną i interfejs użytkownika, lecz nie zmienia to monolitycznego charakteru tego rozwiązania.

Architektura mikroserwisowa charakteryzuje się rozbiciem backendu na niezależne i luźno powiązane ze sobą komponenty, nazywane mikroserwisami. Każdy z nich realizuje ściśle określoną funkcję biznesową i może być wdrażany niezależnie.

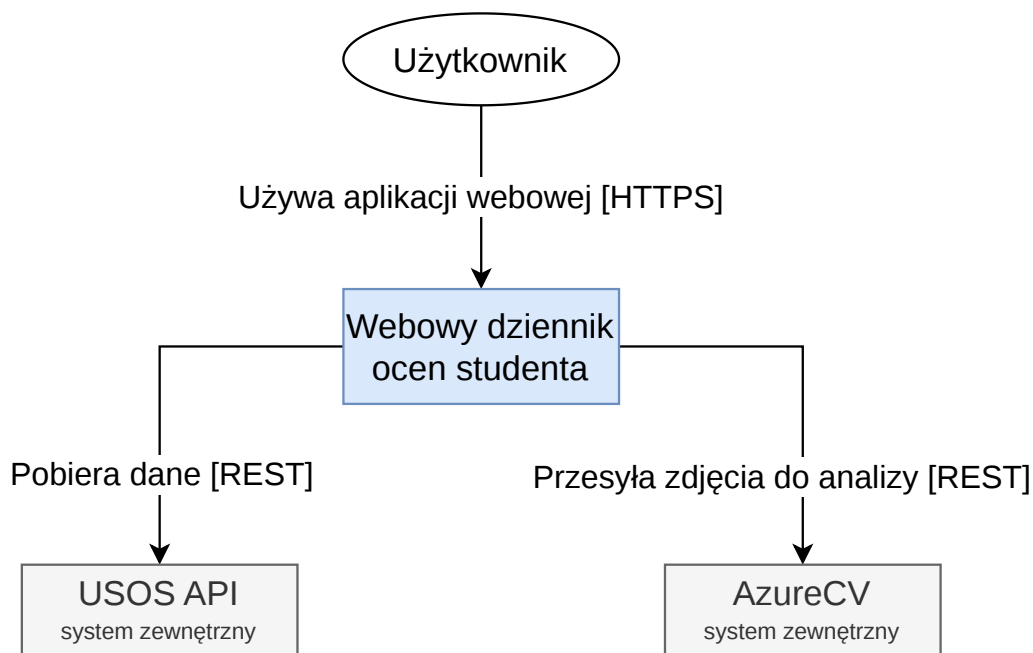
Kryterium	Monolit	Mikroserwisy
Złożoność wdrożenia	Niska – jeden artefakt	Wysoka – wiele niezależnych serwisów
Skalowalność	Skalowanie całego systemu	Niezależne skalowanie serwisów
Izolacja błędów	Błąd może wpłynąć na cały system	Awaria jednego serwisu nie zatrzymuje pozostałych
Komunikacja	Bezpośrednie wywołania metod	Komunikacja sieciowa (REST API)
Testowalność	Prostsze testy integracyjne	Łatwiejsze testy jednostkowe serwisów
Czas wdrożenia zmian	Wymaga przebudowania całości	Niezależne wdrażanie poszczególnych serwisów
Złożoność kodu	Niższa na początku	Wyższa – konieczność obsługi komunikacji

Tabela 1: Porównanie architektury monolitycznej i mikroserwisowej

4.2 Kontekst systemu

Na poniższym rysunku przedstawiono diagram przedstawiający system jako całość na tle jego zewnętrznego środowiska. Stanowi on poziom pierwszy modelu C4. Celem diagramu jest ukazanie granic systemu oraz jego relacji z aktorami i systemami zewnętrznymi, bez wnikania w wewnętrzną strukturę aplikacji.

Diagram kontekstu systemu



Rysunek 1: Diagram kontekstu systemu

Źródło: opracowanie własne z użyciem draw.io

Elementy widniejące na diagramie można podzielić na trzy kategorie: aktorzy, system oraz systemy zewnętrzne. Aktorem jest użytkownik, który komunikuje się z systemem za pośrednictwem przeglądarki internetowej z wykorzystaniem protokołu HTTPS. System, czyli webowy dziennik ocen studenta zapewnia integrację z dwoma systemami zewnętrznymi – USOS oraz Azure Computer Vision – poprzez wysyłanie żądań na ich endpointy z wykorzystaniem protokołu REST.

4.3 Uzasadnienie wyboru architektury mikroserwisowej

Wybór architektury mikroserwisowej podyktowany był kilkoma czynnikami wynikającymi bezpośrednio ze specyfiki projektu.

Pierwszym z nich jest naturalny podział domeny na odrębne obszary funkcjonalne – zarządzanie grupami, ocenami, obecnością, uwierzytelnianie oraz automatyczne ocenianie sprawdzianów. Każdy z nich posiada własną, izolowaną bazę danych. Takie podejście pozwala na niezależne zarządzanie schematem danych w obrębie każdego serwisu oraz eliminuje ryzyko niezamierzonych skutków ubocznych przy modyfikacji schematu współdzielonej bazy.

Istotnym argumentem przemawiającym za architekturą mikroserwisową jest wyraźne rozgraniczenie odpowiedzialności w zakresie komunikacji z serwisami zewnętrznymi. Komunikacja z USOS przebiega tylko i wyłącznie przez serwis autoryzacji, natomiast integracja z Azure Computer Vision jest zadaniem serwisu automatycznej oceny sprawdzianów. Takie podejście ogranicza wpływ ewentualnych zmian w serwisach zewnętrznych do pojedynczego mikroserwisu.

Dodatkową zaletą poczynionego wyboru jest izolacja błędów. Awaria jednego mikroserwisu nie powoduje zatrzymania całego systemu. Przykładowo, awaria serwisu automatycznej oceny sprawdzianów nie przeszkodzi wykładowcy w zarządzaniu ocenami i obecnościami studentów, ponieważ serwisy te działają jako niezależne procesy.

Architektura mikroserwisowa sprzyja również łatwości rozbudowy systemu. Dodanie nowej funkcjonalności nie wymaga ingerencji w istniejące już serwisy, o ile zależności między nimi pozostaną niezmiennione. Rozwijanie istniejących serwisów również jest prostsze ze względu na mniejszą bazę kodu, co przekłada się na krótszy czas analizy jej aktualnego stanu, a tym samym przyspiesza wdrożenie zmian.

4.4 Komponenty systemu

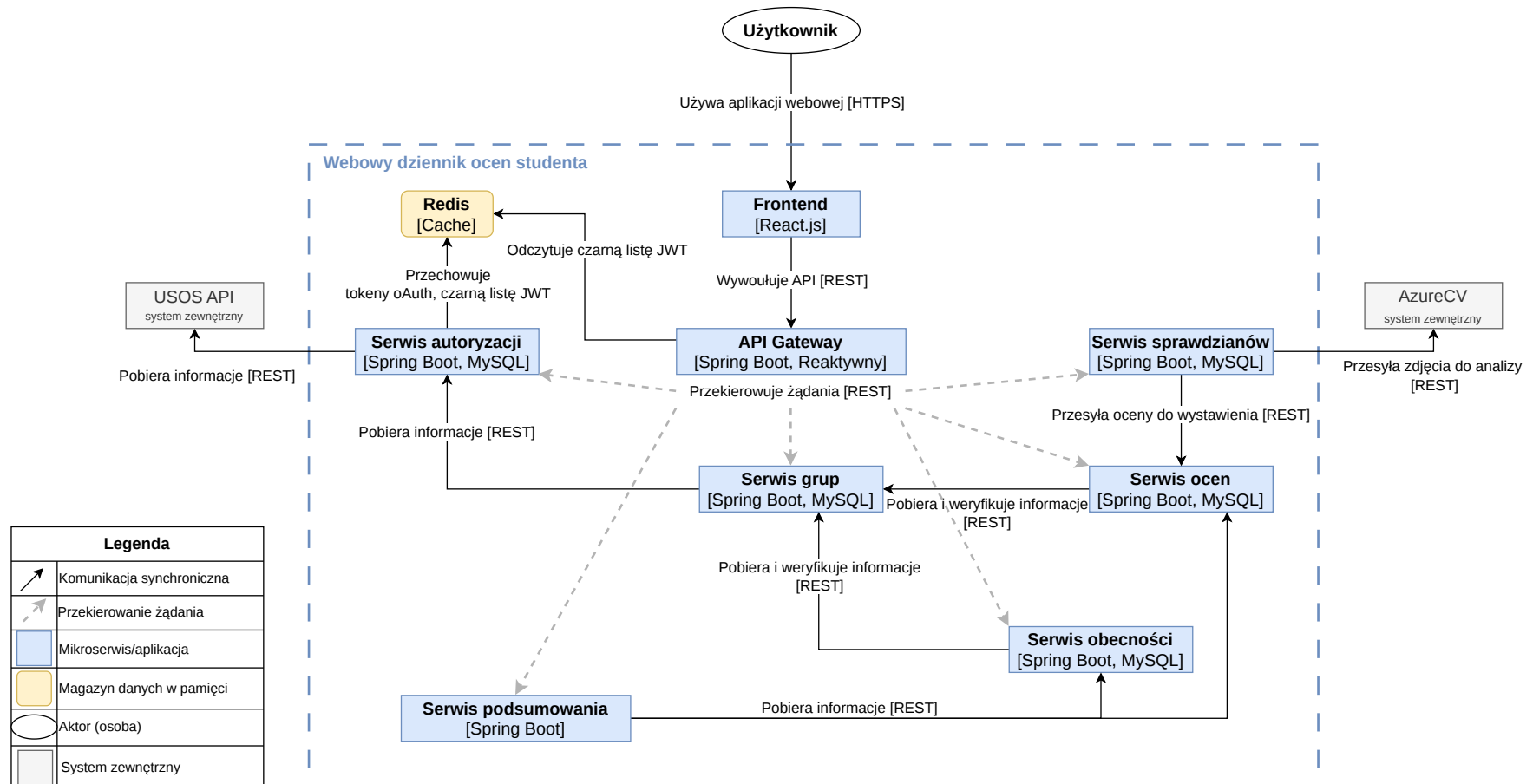
Diagram komponentów systemu znajdujący się na następnej stronie stanowi drugi poziom modelu C4. Przedstawia wewnętrzną strukturę systemu.

Pominięto mechanizm weryfikacji tokenów JWT w celu zachowania czytelności – każdy mikroserwis pełni rolę serwera zasobów¹⁰. Oznacza to, że każdy z nich samodzielnie weryfikuje sygnaturę tokenu JWT, przesłanego w żądaniu, przy użyciu klucza publicznego pobranego z endpointa JWKS¹¹ serwisu autoryzacji.

¹⁰Wzorzec serwera zasobów (*ang. Resource Server Pattern*) – wzorzec architektoniczny polegający na wydzieleniu osobnego komponentu odpowiedzialnego wyłącznie za przechowywanie i udostępnianie chronionych zasobów, odseparowanego od logiki uwierzytelniania i autoryzacji.

¹¹JWKS (*ang. JSON Web Keyset*) – standard definiujący format przechowywania i udostępniania kluczy kryptograficznych w postaci JSON, wykorzystywany do publikowania kluczy publicznych niezbędnych do weryfikacji podpisów tokenów JWT.

Diagram komponentów systemu



Rysunek 2: Diagram kontenerów

Źródło: opracowanie własne z użyciem draw.io

4.5 Role pełnione przez poszczególne komponenty

4.5.1 Frontend

Frontend jest jedynym komponentem, z którym użytkownik wchodzi w interakcję. Zapewnia użytkownikowi intuicyjny interfejs graficzny, za pomocą którego wysyła zapytania do backendu.

4.5.2 API Gateway

Na rysunku (2) można zaobserwować przejście przez system zapytania wysłanego przez użytkownika za pośrednictwem frontendu. Pierwszym przystankiem zawsze jest reaktywny¹² API Gateway, który przekierowuje je do odpowiedniego serwisu wykorzystując jego endpointy. Zanim jednak nastąpi przekierowanie, sprawdza czy JWT nie znajduje się na czarnej liście pobieranej ze współdzielonej z serwisem autoryzacji pamięci cache (Redis). Jeśli token nie znajduje się na czarnej liście, następuje przekierowanie.

4.5.3 Serwis autoryzacji

Serwis autoryzacji pełni w systemie znaczące role – zapewnia bezpieczeństwo, autoryzuje użytkowników oraz integruje się z systemem USOS w celu pobrania danych.

4.5.4 Serwis grup

Kluczowym elementem jest serwis grup – przechowuje dane o grupach zajęciowych oraz aktualnym semestrze pobrane z serwisu autoryzacji. Eksportuje informacje o aktualnym semestrze do serwisów obecności i ocen. Jest też dla nich jedynym źródłem prawdy na temat uprawnień użytkowników w poszczególnych grupach zajęciowych.

4.5.5 Serwisy ocen i obecności

Oba wymienione mikroservisy realizują operacje CRUD¹³ na zarządzanych przez siebie zasobach. Pobierają informacje o aktualnym semestrze z serwisu grup oraz zapisują je w swoich bazach danych. Przy zarządzaniu ocenami i obecnościami, komunikują się z serwisem grup w celu weryfikacji uprawnień użytkownika, który wysłał zapytanie.

4.5.6 Serwis podsumowania

Mikroservis podsumowania jest całkowicie zależny od serwisów ocen i obecności. Jego jedynym zadaniem jest pobranie danych z obu wcześniej wymienionych mikroservisów i wygenerowanie czytelnego podsumowania pracy studentów na zajęciach.

¹²Reaktywny model programowania (*ang. Reactive Programming*) – paradygmat programowania oparty na asynchronicznym przetwarzaniu strumieni danych i propagacji zmian, umożliwiający nieblokującą obsługę wielu zapytań jednocześnie przy minimalnym zużyciu zasobów. W kontekście API Gateway pozwala na wydajną obsługę dużej liczby równoczesnych połączeń.

¹³CRUD (*ang. Create, Read, Update, Delete*, pol. utwórz, odczytaj, zaktualizuj, usuń) – cztery podstawowe operacje wykonywane na zasobach w systemach informatycznych, odpowiadające tworzeniu, odczytywaniu, aktualizowaniu oraz usuwaniu danych.

4.5.7 Serwis sprawdzianów

Serwis sprawdzianów jest wyróżniającym się komponentem w systemie – zapewnia on funkcjonalność automatycznego oceniania sprawdzianów z wykorzystaniem integracji z usługą Azure Computer Vision. Odpowiedzialność za wystawianie ocen jest oddelegowana do serwisu ocen.

4.6 Izolacja danych w mikroserwisach

Jedną z kluczowych zasad architektury mikroserwisowej jest izolacja danych – każdy mikroservis posiada własną, niezależną bazę danych, do której żaden inny serwis nie ma bezpośredniego dostępu. Wymiana danych między serwisami odbywa się wyłącznie za pośrednictwem zdefiniowanych interfejsów API, co zapewnia luźne powiązanie komponentów systemu. Umożliwia także niezależne zarządzanie schematem danych w obrębie każdego serwisu.

Relacje między danymi należącymi do różnych mikroservisów mają charakter logiczny, nie fizyczny. Zamiast kluczy obcych łączących tabele w różnych bazach danych, serwisy przechowują jedynie identyfikator obiektów należących do innych serwisów. Przykładowo, serwis ocen nie posiada fizycznej relacji z tabelą studentów w serwisie grup. Przechowuje jedynie identyfikator studenta oraz jednostki zajęciowej, a spójność danych jest zapewniana na poziomie logiki aplikacji.

4.7 Przyjęte wzorce na poziomie serwisu

Przyjęto wzorzec architektury warstwowej, dzieląc kod na warstwę prezentacji, logiki biznesowej oraz dostępu do danych. W serwisach posiadających własną bazę danych dodatkowo wykorzystano wzorce Repository[4] oraz DTO[5]. We wszystkich mikroservisach zastosowano wzorce Dependency Injection[6] oraz Singleton[7].

5 Przegląd technologii wykorzystanych do budowy aplikacji

Niniejszy rozdział zawiera przegląd technologii zastosowanych w projekcie. Dla każdej z nich przedstawiono krótką charakterystykę, opis zastosowania w projekcie oraz uzasadnienie wyboru.

5.1 Spring Boot

5.1.1 Charakterystyka

Spring Boot to framework¹⁴ aplikacyjny na platformie Java, będący rozszerzeniem frameworka Spring. Jego główną zaletą jest zasada konwencji ponad konfigurację. Dostarcza domyślną konfigurację dla większości komponentów, znacząco redukując ilość kodu konfiguracyjnego wymaganego do uruchomienia aplikacji na zewnętrznym serwerze – aplikacja jest uruchamiana jako samodzielny proces.

5.1.2 Zastosowanie w projekcie

W projekcie wykorzystano Spring Boot jako podstawowy framework do budowy warstwy backendowej. Każdy mikroservis stanowi niezależny projekt, posiadający własną konfigurację zależności. Wszystkie serwisy wykorzystują Spring Boot w wersji 3.4.12, za wyjątkiem serwisu autoryzacji który wykorzystuje wersję 3.5.7. Każdy z nich został zaprogramowany w języku Java w wersji 21.

5.1.3 Uzasadnienie wyboru

Spring Boot jest szczególnie dobrze przystosowany do budowy aplikacji mikroservisowych. Wbudowany serwer webowy umożliwia uruchamianie każdego serwisu jako niezależnego procesu, a ekosystem Spring Cloud dostarcza gotowe rozwiązania dla typowych wyzwań zastosowanej w projekcie architektury.

Framework wspiera wzorce projektowe na poziomie serwisu, opisane w poprzednim rozdziale. Architektura warstwowa realizowana jest przez adnotacje `@RestController`, `@Service` oraz `@Repository`, które jednoznacznie określają przynależność klasy do odpowiedniej warstwy. Wzorzec Dependency Injection jest fundamentem działania kontenera Spring – zależności między komponentami wstrzykiwane są przez konstruktor, co eliminuje konieczność ręcznego tworzenia instancji obiektów. Każdy komponent zarządzany przez kontener Spring jest domyślnie Singletonem, co oznacza, że przez cały czas działania aplikacji istnieje tylko jedna jego instancja. Wzorzec Repository jest wspierany bezpośrednio przez moduł Spring Data JPA.

¹⁴framework – gotowa struktura oprogramowania dostarczająca zestaw narzędzi, bibliotek i konwencji, na bazie której programista buduje aplikację.

5.2 React

5.2.1 Charakterystyka

React to biblioteka JavaScript służąca do budowy interfejsów użytkownika, stworzona i rozwijana przez korporację Meta. Opiera się na komponentowym modelu budowy UI. Oznacza to, że interfejs jest dekomponowany na niezależne komponenty wielokrotnego użytku. Każdy z nich zarządza własnym stanem i logiką renderowania. React wykorzystuje wirtualny DOM (ang. *Vitruual DOM*) – reprezentację drzewa elementów przechowywaną w pamięci. Jest on synchronizowany z rzeczywistym drzewem DOM przeglądarki tylko w zakresie faktycznie zmienionych elementów, co przekłada się na wydajność renderowania.

5.2.2 Zastosowanie w projekcie

Frontend aplikacji został zrealizowany jako Single Page Application¹⁵ z wykorzystaniem React w wersji 19 i React Router DOM w wersji 7. Do stylizowania intefesju użyto Tailwind CSS w wersji 4. Aplikacja napisana jest w TypeScript w wersji 5.9. Projekt budowany jest przy użyciu Vite w wersji 7.

5.2.3 Uzasadnienie wyboru

React jest jedną z najpopularniejszych bibliotek do budowy interfejsów użytkownika, co przekłada się na dojrzały ekosystem z bogatą dokumentacją oraz szeroką dostępnością gotowych rozwiązań. Komponentowy model budowy UI sprzyja ponownemu wykorzystaniu kodu, co jest szczególnie istotne w przypadku powtarzających się elementów interfejsu. Wybór TypeScript zapewnia statyczne typowanie, redukując ryzyko błędów na etapie kompilacji.

5.3 MySQL

5.3.1 Charakterystyka

MySQL to relacyjny system zarządzania bazą danych oparty na języku SQL, rozwijany przez Oracle. Dane przechowywane w tabelach powiązanych ze sobą relacjami, co zapewnia ich integralność i spójność. MySQL jest jednym z najpopularniejszych systemów bazodanowych na świecie, cechującym się wysoką wydajnością, niezawodnością oraz szerokim wsparciem społeczności.

5.3.2 Zastosowanie w projekcie

MySQL pełni rolę silnika bazodanowego dla serwisów posiadających własną warstwę persystencji. W projekcie wykorzystano wersję 8.

¹⁵Single Page Application (SPA) – typ aplikacji webowej, w której cała zawartość ładowana jest jednorazowo przy pierwszym żądaniu, a kolejne interakcje użytkownika obsługiwane są dynamicznie po stronie klienta bez przeładowania strony, co zapewnia płynniejsze i szybsze działanie interfejsu.

5.3.3 Uzasadnienie wyboru

MySQL został wybrany ze względu na dojrzałość technologii, szeroką dokumentację oraz bezpośrednie wsparcie ze strony modułu Spring Data JPA i Hibernate – biblioteki ORM¹⁶ wchodzącej w skład ekosystemu Spring Boot.

5.4 Redis

5.4.1 Charakterystyka

Redis to magazyn danych typu klucz-wartość przechowujący dane w pamięci operacyjnej, co zapewnia bardzo niskie czasy dostępu w porównaniu do tradycyjnych baz danych.

5.4.2 Zastosowanie w projekcie

Redis pełni w systemie rolę współdzielonego magazynu danych dla serwisu autoryzacji oraz API Gateway. Wykorzystana została wersja 7.

5.4.3 Uzasadnienie wyboru

Redis został użyty w projekcie ze względu na przechowywanie danych w pamięci operacyjnej. Czyni go to idealnym rozwiązaniem dla danych wymagających szybkiego dostępu oraz późniejszego usunięcia. Ma wbudowane wsparcie dla wygasania kluczy. Dodatkowym argumentem przemawiającym za tym wyborem jest bezpośrednie wsparcie w ekosystemie Spring Boot poprzez moduł Spring Data Redis, który dostarcza gotową abstrakcję nad operacjami danych.

5.5 Azure Computer Vision

5.5.1 Charakterystyka

Azure Computer Vision to usługa chmurowa firmy Microsoft udostępniana w ramach platformy Azure, oferująca funkcjonalności analizy obrazów. W projekcie wykorzystano funkcjonalność OCR, umożliwiającą ekstrakcję tekstu ze zdjęć.

5.5.2 Zastosowanie w projekcie

Usługa wykorzystywana jest przez serwis sprawdzianów do analizy zdjęć przesłanych przez wykładowcę. Komunikacja z Azure Computer Vision została zrealizowana za pośrednictwem klienta HTTP, wysyłając zapytania bezpośrednio na endpointy udostępnione przez usługę.

¹⁶ORM (ang. *Object-Relational Mapping*, pol. mapowanie obiektowo-relacyjne) – technika programowania polegająca na automatycznym odwzorowaniu obiektów aplikacji na tabele relacyjnej bazy danych, eliminując konieczność ręcznego pisania zapytań SQL.

5.5.3 Uzasadnienie wyboru

Wybór został podjęty ze względu na wysoką skuteczność rozpoznawania tekstu oraz prostotę integracji poprzez REST API. Azure Computer Vision, jako usługa chmurowa, nie wymaga utrzymania własnej infrastruktury do przetwarzania obrazów, co znacząco upraszcza architekturę systemu. Alternatywą było uruchomienie lokalnego silnika OCR, jednak wiązałoby się to z koniecznością jego konfiguracji, utrzymania oraz dostosowania modelu do rozpoznawania pisma odręcznego, co wykraczało poza zakres projektu.

5.6 Docker

5.6.1 Charakterystyka

Docker to platforma do konteneryzacji aplikacji. Kontener jest izolowaną jednostką wdrożeniową zawierającą aplikację wraz ze wszystkimi jej zależnościami. Gwarantuje to spójne środowisko wdrożeniowe niezależne od infrastruktury hosta. Narzędzie Docker Compose umożliwia konfigurację środowiska wielu kontenerów w sposób deklaratywny.

5.6.2 Zastosowanie w projekcie

Wszystkie komponenty systemu zostały skonteneryzowane i są uruchamiane wspólnie przy użyciu Docker Compose. Obrazy mikroservisów budowane są przy pomocy wtyczki Jib Maven Plugin, która generuje obraz Docker bez konieczności tworzenia pliku DockerFile.

5.6.3 Uzasadnienie wyboru

Docker został wybrany ze względu na prostotę wdrożenia oraz izolację komponentów systemu. Konteneryzacja eliminuje problem różnic środowiskowych między różnymi maszynami. Oznacza to, że system może być bezproblemowo wdrożony na każdym serwerze z zainstalowanym środowiskiem Docker.

6 Implementacja backendu

6.1 Współdzielone zależności

Każdy mikroservis jest niezależnym projektem, posiadającym własną konfigurację zależności. Mimo tego, analiza plików konfiguracyjnych poszczególnych serwisów wykazała szereg zależności powtarzających się w obrębie całego systemu. Świadczy to o spójnym podejściu do implementacji.

Powtarzające się zależności:

- Spring Security + OAuth2 Resource Server – bezpieczeństwo i weryfikacja JWT,
- Actuator – monitorowanie metryk aplikacji takich jak użycie pamięci, liczba żądań czy czasy odpowiedzi,
- Zalando Logbook – logowanie żądań HTTP,
- Spring Cloud Circuit Breaker – implementacja wzorca Circuit Breaker[8],
- Project Lombok – redukcja powtarzalnego kodu, który jest charakterystyczny dla języka Java,
- Spring Data JPA – warstwa dostępu do danych zapewniająca obiektowo-relacyjne mapowanie encji na tabele bazy danych oraz abstrakcję nad Hibernate poprzez interfejs `JpaRepository`,
- MySQL Connector/J – łączność z bazą danych,
- Spring Cloud Open Feign – klient HTTP wykorzystywany do komunikacji między serwisami,
- Spring Validation – walidacja danych

6.2 Bezpieczeństwo

Bezpieczeństwo systemu przetwarzającego dane osobowe studentów jest wymaganiem kluczowym, wynikającym zarówno z przepisów RODO, jak i samej natury aplikacji.

6.2.1 JWT

JWT jest otwartym standardem definiującym sposób bezpiecznego przesyłania informacji między stronami, wykorzystując format JSON. Token składa się z trzech części: nagłówka, ładunku oraz podpisu. Nagłówek zawiera informacje o użytym algorytmie podpisu, ładunek przechowuje twierdzenia (ang. *claims*) dotyczące użytkownika, natomiast podpis zapewnia integralność.

W projekcie JWT pełni rolę bezstanowego mechanizmu uwierzytelniania użytkowników. Po pomyślnym zalogowaniu użytkownika za pośrednictwem OAuth¹⁷ USOS, serwis autoryzacji generuje token JWT podpisany kluczem prywatnym algorytmu RSA[9].

¹⁷OAuth – otwarty standard autoryzacji umożliwiający aplikacjom trzecim uzyskanie ograniczonego dostępu do zasobów użytkownika bez ujawniania jego danych uwierzytelniających.

Token zawiera następujące twierdzenia: identyfikator użytkownika, imię, rolę, czas wygaśnięcia i czas wydania.

6.2.2 Role i dostęp

W systemie zdefiniowano dwie role użytkowników określające poziom dostępu do zasobów oraz uprawnienia do zarządzania nimi – student i wykładowca. W praktyce oznacza to, że dostęp do poszczególnych endpointów jest zależny od posiadanej roli. Niektóre z nich są współdzielone, a rola użytkownika jest sprawdzana z poziomu logiki biznesowej – na jej podstawie zwracana jest odpowiednia odpowiedź.

6.2.3 Implementacja

Konfiguracja bezpieczeństwa opiera się na modułach Spring Security oraz OAuth2 Resource Server. Każdy mikroservis posiada dedykowaną klasę konfiguracji `SecurityConfig`, która definiuje reguły dostępu do endpointów oraz konfiguruje weryfikację tokenów JWT. Jedyne niestrzeżone endpointy należą do procesu autoryzacji z USOS oraz metryk udostępnianych przez Actuator.

Weryfikacja tokenu odbywa się automatycznie, poprzez pobranie klucza publicznego z endpointu JWKS udostępnianego przez serwis autoryzacji. Zgodnie z zasadą Zero Trust¹⁸, każdy mikroservis samodzielnie weryfikuje podpis tokenu JWT, nie polegając na założeniu, że żądanie zostało uwierzytelnione przez inny komponent systemu.

Rola użytkownika jest determinowana z twierdzenia zawartego w tokenie i mapowana na uprawnienia Spring Security. Umożliwia to kontrolę dostępu na poziomie metod przy użyciu adnotacji `@PreAuthorize`.

6.3 Komunikacja między mikroservisami

Komunikacja między mikroservisami jest realizowana za pośrednictwem biblioteki Spring Cloud OpenFeign, która umożliwia deklaratywne definiowanie klientów HTTP przy użyciu interfejsów z odpowiednimi adnotacjami. Zamiast ręcznego budowania żądań HTTP, wystarczy zdefiniować interfejs opisujący docelowy endpoint i założony format odpowiedzi, a Feign automatycznie generuje jego implementację.

Kluczowym elementem konfiguracji jest filtr żądań zdefiniowany w klasie `FeignJwtConfig`. Przy każdym wywołaniu, odczytuje nagłówek `Authorization` z bieżącego żądania HTTP i dołącza go do wychodzącego żądania. Dzięki temu token JWT użytkownika jest propagowany między serwisami.

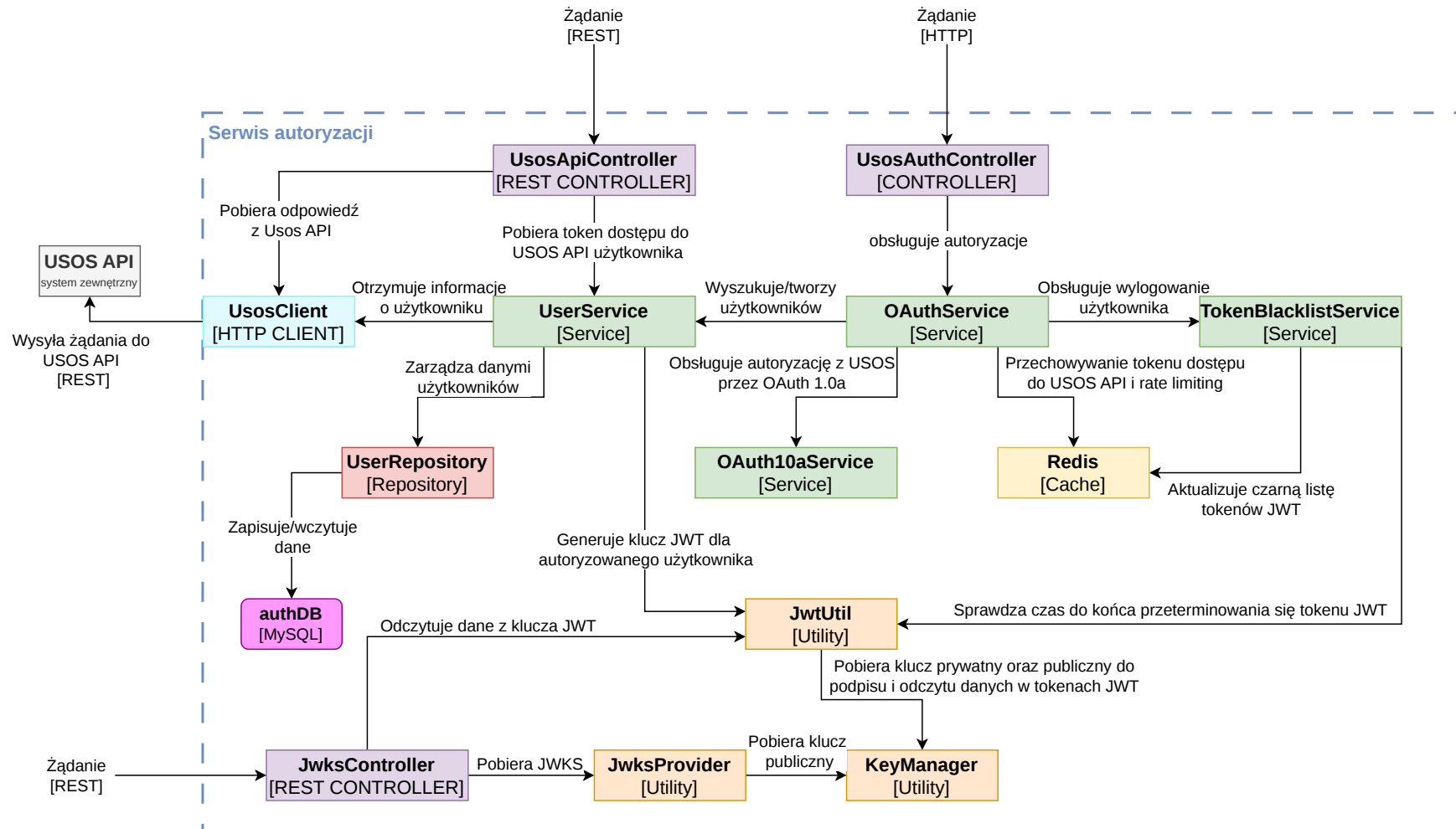
6.4 Serwis autoryzacji

6.4.1 Diagram komponentów

Na następnej stronie przedstawiony został diagram komponentów serwisu autoryzacji.

¹⁸Zero Trust (pol. zasada braku zaufania) – model bezpieczeństwa zakładający brak domyślnego zaufania do żadnego komponentu systemu, niezależnie od jego lokalizacji w sieci.

Diagram komponentów serwisu — Serwis autoryzacji



Rysunek 3: Diagram komponentów serwisu autoryzacji

Źródło: opracowanie własne z użyciem draw.io

Diagram (3) ma na celu zaprezentować zachodzące między nimi interakcje oraz podział wykonywanych przez nie zadań. Stanowi on poziom trzeci modelu C4. Zastosowane w nim nazewnictwo odpowiada temu w bazie kodu mikroserwisu

6.4.2 Proces autoryzacji wewnątrz systemu

Jak można zaobserwować na diagramie z rysunku (3), proces autoryzacji z USOS rozpoczyna się w kontrolerze `UsosAuthController`. Deleguje on obsługę tego procesu komponentowi `OAuthService`, który obsługuje proces uwierzytelniania użytkownika za pomocą protokołu OAuth 1.0a z systemem USOS. Po pomyślnym uwierzytelnieniu wyszukuje lub tworzy użytkownika w bazie danych, a następnie zleca wygenerowanie tokenu JWT komponentowi `JwtUtil`, który wykorzystuje klucze RSA zarządzane przez `KeyManager`. Wygenerowany token jest przekazywany do frontendu przez kontroler za pomocą HTTP Only Cookies¹⁹.

Wylogowanie obsługiwane jest przez `TokenBlackListService`, który zapisuje unieważniony token JWT w magazynie danych Redis. Kontroler czyści ciasteczko uwierzytelniające po stronie klienta. Taka strategia uniemożliwia dalsze wykorzystanie tokenu zarówno po stronie przeglądarki, jak i serwera, zachowując spójność między obiema warstwami.

6.4.3 JWKS

Komponent `JwksProvider` opakuje klucz publiczny w format `RSAPublicKey` biblioteki Nimbus JOSE. Kontroler `JwksController` udostępnia klucz publiczny pod endpointem `/jwks.json` w formacie `JWKS`. Każdy mikroserwis pobiera go automatycznie przy starcie aplikacji na podstawie właściwości `jwk-set-uri` zdefiniowanej w pliku konfiguracyjnym.

6.4.4 Proces Autoryzacji z USOS API

Autoryzacja z USOS API odbywa się za pośrednictwem OAuth 1.0a. Jest to protokół autoryzacji umożliwiający aplikacjom trzecim dostęp do zasobów użytkownika bez konieczności udostępniania jego danych logowania. W przeciwieństwie do nowszego OAuth 2.0, wersja 1.0a opiera się na kryptograficznym podpisywaniu żądań przy użyciu sekretu konsumenta oraz tokenu dostępu, co eliminuje konieczność stosowania szyfrowanego połączenia jako jedyne zabezpieczenie.

Należy zaznaczyć, że OAuth 1.0a jest protokołem uznawanym za przestarzały. Większość współczesnych systemów migruje na OAuth 2.0, który upraszcza proces autoryzacji i lepiej sprawdza się w środowiskach mobilnych i chmurowych. Wykorzystanie tego protokołu w projekcie nie wynika z preferencji technicznych, lecz z ograniczeń narzuconych przez USOS API, które nie wspiera nowszego standardu.

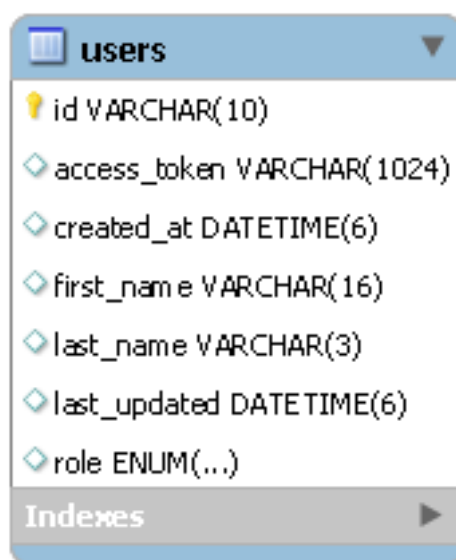
Autoryzacja przebiega w trzech etapach[10]. W pierwszym etapie aplikacja wykonuje dwustronne wywołanie API do endpointu `services/oauth/request_token` w celu pobrania tymczasowego tokenu żądania oraz jego sekretu. Następnie użytkownik jest

¹⁹HTTP Only Cookie – plik cookie z flagą `HttpOnly`, który nie jest dostępny z poziomu JavaScript.

przekierowywany na stronę autoryzacji USOS, gdzie po zalogowaniu wyraża zgodę na udostępnienie danych aplikacji. Po udzieleniu zgody USOS przekierowuje użytkownika z powrotem do serwisu autoryzacji, dołączając do adresu URL kod weryfikacyjny OAuth. W ostatnim etapie aplikacja wykonuje trójstronne wywołanie API, podpisując żądanie sekretem konsumenta oraz sekretem tokenu żądania, w zamian otrzymując docelowy token dostępu oraz sekret tokenu dostępu. Od tego momentu aplikacja posiada wystarczające zasoby do wywołania API w imieniu użytkownika.

Implementacja przepływu OAuth 1.0a realizowana jest przy użyciu biblioteki ScribeJava w wersji 8.3.3. Jest to biblioteka dedykowana obsłudze protokołów OAuth w różnych wersjach. Dostarcza gotowe abstrakcje nad procesem podpisywania żądań oraz zarządzania tokenami, eliminując konieczność ręcznej implementacji mechanizmów kryptograficznych wymaganych przez ten protokół.

6.4.5 Schemat bazy danych



Rysunek 4: Schemat bazy danych serwisu autoryzacji

Źródło: opracowanie własne z użyciem MySQL Workbench

Nowi użytkownicy są zapisywani w tabeli przedstawionej na rysunku (4). Identyfikator użytkownika jest tożsamy z tym, który ma przypisany w systemie USOS. Kolumna access token przechowuje zaszyfrowany token dostępu do USOS API tylko dla wykładowców oraz tylko do momentu wylogowania. Czas ważności tego tokenu wynosi dwie godziny i jest taki sam, jak w przypadku JWT. Przyjęta strategia zapewnia jednolite zarządzanie sesjami w systemie.

Token dostępu jest szyfrowany algorytmem AES-256[11] w trybie GCM[12] przed zapisem do bazy danych. Tryb ten zapewnia zarówno poufność danych jak i ich integralność poprzez dołączenie znacznika uwierzytelniającego do zaszyfrowanych danych. Dla każdej operacji szyfrowania generowany jest losowy wektor inicjalizacyjny o długości 96

bitów, który jest dołączany do zaszyfrowanych danych przed zapisem. Dzięki temu każde szyfrowanie tego samego tekstu daje inny wynik, co eliminuje podatność na ataki krypto-analityczne.

6.4.6 Komunikacja z USOS API

Komunikacja z USOS API jest realizowana za pośrednictwem webowego klienta – HTTP WebClient. Każde żądanie wysyłane do USOS API musi być podpisane zgodnie z protokołem OAuth 1.0a. Odpowiada za to filtr skonfigurowany w klasie konfiguracyjnej klienta HTTP. Przechwytuje on każde wychodzące żądanie, odczytuje token dostępu oraz sekret tokenu przekazane w nagłówkach **X-User-Token** i **X-User-Secret**. Następnie podpisuje żądanie i dołącza nagłówek **Authorization** przed jego wysłaniem.

Kontroler `UsosApiController` udostępnia endpointy, pozwalające pobrać informacje z systemu USOS. Klasa `UsosClient` implementuje cztery metody odpowiedzialne za import danych z API:

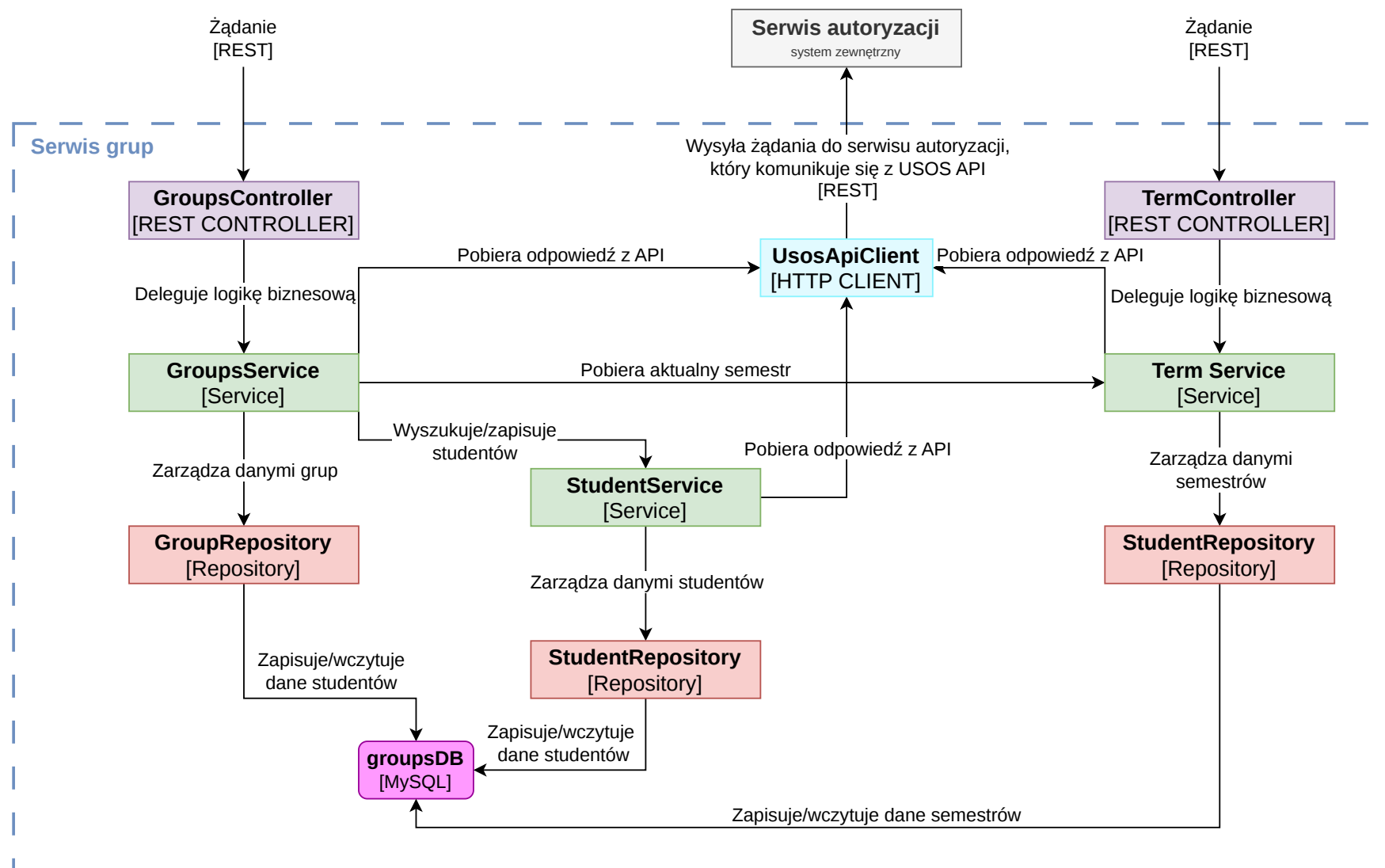
- `getUserDetails` – pobiera podstawowe dane użytkownika konieczne do utworzenia konta w aplikacji,
- `getActiveTerm` – pobiera aktywny semestr na podstawie bieżącej daty,
- `getGroups` – pobiera listę grup zajęciowych prowadzonych przez wykładowcę wraz z listą uczestników,
- `getUserInfo` – pobiera szczegółowe dane pojedynczego studenta na podstawie jego identyfikatora, w tym numer indeksu;

6.5 Serwis grup

6.5.1 Diagram komponentów

Na kolejnej stronie przedstawiono diagram komponentów serwisu grup.

Diagram komponentów serwisu — Serwis grup



Rysunek 5: Diagram komponentów serwisu grup

Źródło: opracowanie własne z użyciem draw.io

6.5.2 Pobieranie danych

Serwis grup nie komunikuje się bezpośrednio z USOS API. Jedynym punktem integracji z tym systemem zewnętrznym jest serwis autoryzacji. Komponent `UsosApiClient` pokazany na rysunku (5) jest klientem Feign, który wysyła żądania do serwisu autoryzacji, w celu pobrania danych zajęciowych przeprowadzanych przez wykładowcę oraz aktualnym semestrze. Takie podejście centralizuje integrację z USOS API w jednym miejscu.

6.5.3 Grupy zajęciowe

Serwis grup za pośrednictwem kontrolera `GroupsController` udostępnia dwa rodzaje endpointów – przeznaczone dla użytkowników oraz używane wyłącznie przez inne mikroservisy. Endpoint `/api/update` umożliwia wykładowcy synchronizację swoich grup zajęciowych z danymi pobranymi z serwisu autoryzacji. Kontroler deleguje logikę biznesową do `GroupsService`, który podczas synchronizacji wywołuje metodę `findOrCreateParticipants` należącą do `StudentService`. Metoda ta wyszukuje studenta w bazie danych lub tworzy nowy rekord na podstawie informacji pobranych z serwisu autoryzacji.

Oprócz wcześniej wymienionego punktu końcowego `/api/update`, kontroler oferuje endpointy pozwalające użytkownikom i wykładowcom przeglądać swoje grupy. Dostęp do bardziej szczegółowych informacji takich jak lista uczestników zarezerwowany jest jedynie dla wykładowców prowadzących dane grupy. Pozostałe endpointy nie są przeznaczone dla użytkowników frontendu – udostępniają funkcjonalność weryfikacji uprawnień wykładowcy, wykorzystywaną przez serwisy ocen i obecności.

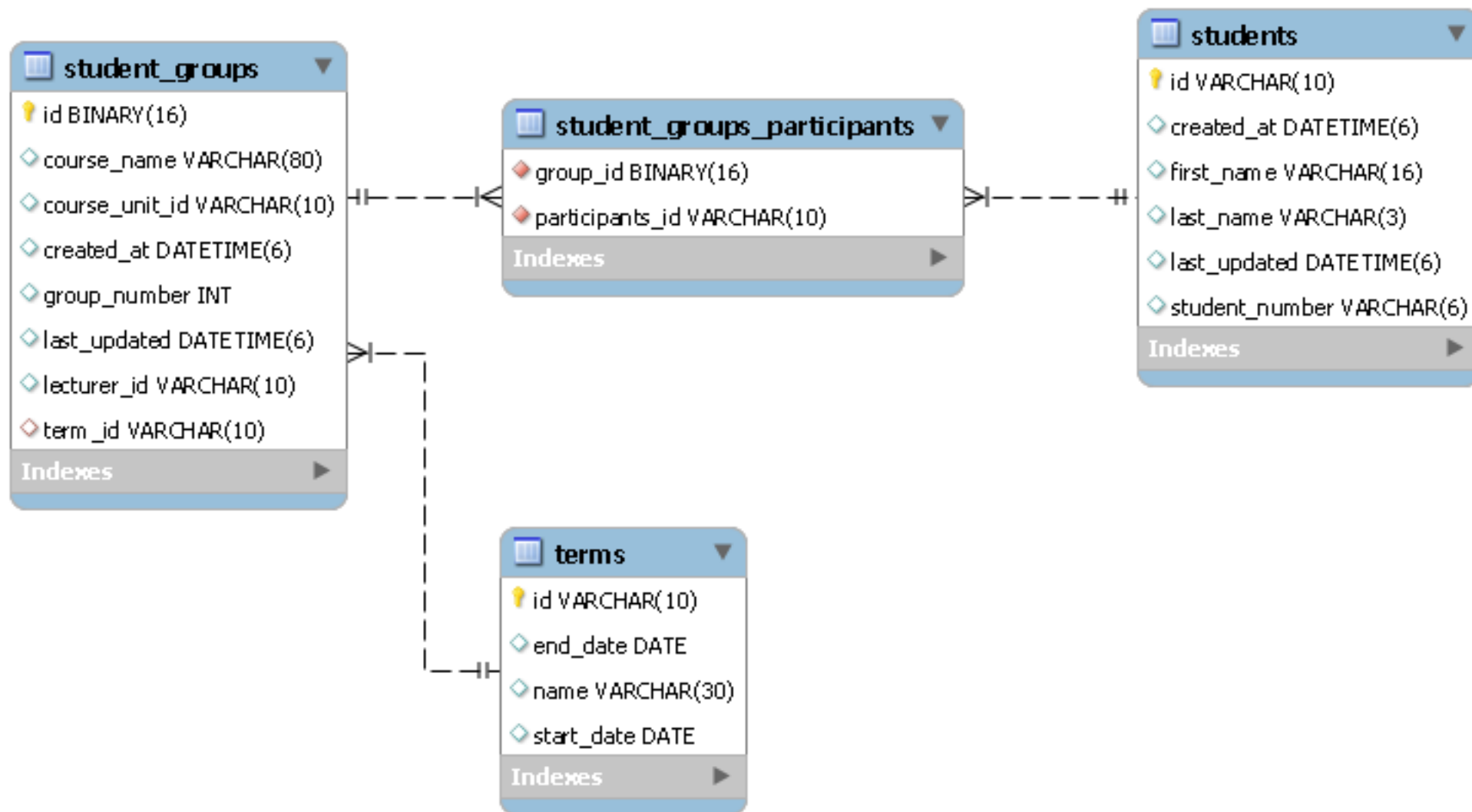
6.5.4 Aktualny semestr

Aktywny semestr pobierany jest za pośrednictwem `TermService`. Stosuje on mechanizmy cache'owania w pamięci operacyjnej. Przy każdym wywołaniu sprawdzane jest czy przechowywany semestr dalej jest aktualny. Jeśli tak, dane są zwracane bezpośrednio z pamięci bez konieczności odpytywania bazy danych. W przeciwnym razie aktywny semestr jest poszukiwany w bazie danych. Jeśli nie zostanie jednak znaleziony, pobierany jest z serwisu autoryzacji i zapisywany.

Kontroler `TermController` udostępnia informacje o aktywnym semestrze pozostałym mikroservisom za pośrednictwem endpointu `/api/activeTerm`.

6.5.5 Schemat bazy danych

Na kolejnej stronie przedstawiony został schemat bazy danych serwisu grup. Do prezentacji relacji między tabelami wykorzystano notację Crow's Foot[13].



Rysunek 6: Schemat bazy danych serwisu grup

Źródło: opracowanie własne z użyciem MySQL Workbench

Schemat (6) przedstawia cztery tabele:

- **student_groups** – przechowuje dane grup zajęciowych,
- **students** – przechowuje dane studentów,
- **students_group_participants** – łączy grupy z ich uczestnikami,
- **terms** – przechowuje dane o semestrach;

Relacja między grupami a studentami jest typu wiele-do-wielu i realizowana jest przez tabelę pośrednią **student_groups_participants**. Między tabelami semestrów i grup zachodzi relacja typu jeden-do-wielu. Jest ona jednostronna.

Identyfikatory studentów oraz semestrów pochodzą bezpośrednio z systemu USOS. Wyjątkiem jest identyfikator tabeli grup, której klucz główny jest generowany jako UUID²⁰ po stronie aplikacji. Unikalność grupy zajęciowej w obrębie systemu wyznaczona jest jednak przez kombinację kolumn **course_unit_id** oraz **group_number**, które jednoznacznie identyfikują grupę w systemie USOS.

6.6 API Gateway

6.6.1 Routing

API Gateway zrealizowane jest przy użyciu Spring Cloud Gateway w oparciu o reaktywny model przetwarzania żądań. Konfiguracja routingu definiuje sześć tras odpowiadających poszczególnym mikroserwisom w systemie. Każde żądanie przychodzące pod ścieżkę `/gradebook/<nazwa_serwisu>/**` jest przepisywane i przekierowywane do właściwego mikroserwisu. Do każdego wychodzącego żądania dołączany jest nagłówek **X-Gateway-Time** zawierający znacznik czasowy jego przetworzenia.

6.6.2 Filtry

Gateway stosuje kilka filtrów globalnych. Filtr **JwtToCookieHeaderFilter** odpowiada za ekstrakcję tokenu JWT z ciasteczka **AUTH_TOKEN**. Dzięki temu frontend nie musi samodzielnie zarządzać nagłówkami wysyłanych żądań. Filtr **JwtBlacklistFilter** sprawdza czy token nie znajduje się na czarnej liście w Redis przed przekierowaniem żądania. W przypadku unieważnionego tokenu żądanie jest odrzucane ze statusem **401 Unauthorized**.

6.6.3 Odporność na błędy

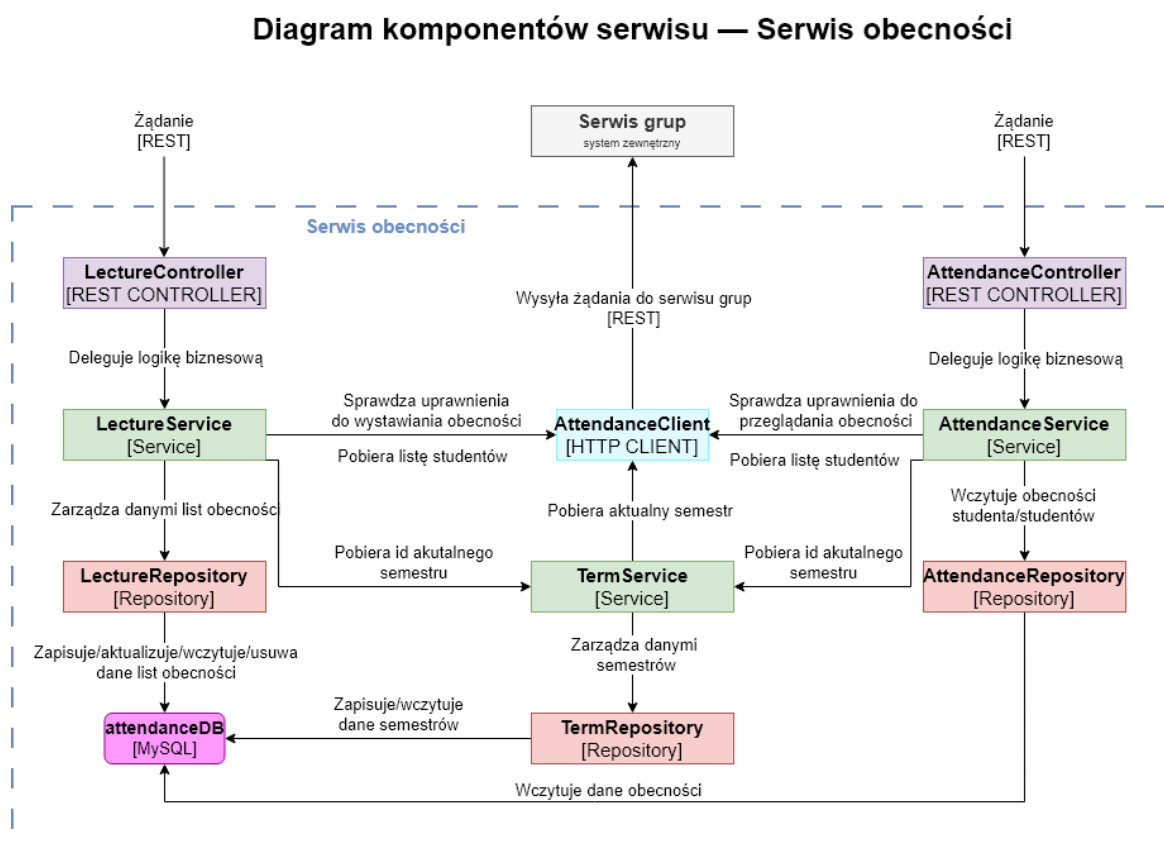
Każda trasa wyposażona jest w mechanizm **Circuit Breaker** oraz – z wyjątkiem serwisu autoryzacji – mechanizm automatycznego ponawiania żądań. Ponowne próby wykonywane są wyłącznie dla metody **GET**. W przypadku niedostępności serwisu żądanie przekierowywane jest do kontrolera **FallbackController**, który zwraca komunikat o

²⁰UUID (ang. *Universally Unique Identifier*, pol. uniwersalny unikalny identyfikator) – 128-bitowy identyfikator generowany w sposób gwarantujący jego unikalność bez konieczności centralnej koordynacji.

niedostępności usługi. Dodatkowo skonfigurowany został rate limiter²¹ oparty na Redis, który ogranicza maksymalną liczbę żądań na sekundę, identyfikując użytkowników na podstawie adresu IP.

6.7 Serwis obecności

6.7.1 Diagram komponentów



Rysunek 7: Diagram komponentów serwisu obecności

Źródło: opracowanie własne z użyciem draw.io

6.7.2 Zarządzanie listami obecności

Serwis obecności udostępnia funkcjonalność zarządzania listami obecności za pośrednictwem kontrolera **LectureController**. Podczas tworzenia nowej listy obecności przez wykładowcę **LectureService** pobiera listę identyfikatorów studentów z serwisu grup, weryfikując przy tym jednocześnie czy prowadzi on daną grupę. W tym procesie dla każdego studenta tworzony jest rekord obecności z domyślnym statusem nieobecny, który może zostać nadpisany wartością przekazaną w żądaniu. Dopuszczane są trzy wartości: obecny, nieobecny i usprawiedliwiony.

²¹Rate limiter (pol. ogranicznik liczby żądań) – mechanizm kontrolujący liczbę żądań wysyłanych do systemu w określonym przedziale czasowym, chroniący przed przeciążeniem serwera oraz atakami typu brute force.

Serwis obsługuje pełen zestaw operacji CRUD na listach obecności. Każda operacja modyfikująca dane poprzedzona jest weryfikacją uprawnień wykładowcy w serwisie grup.

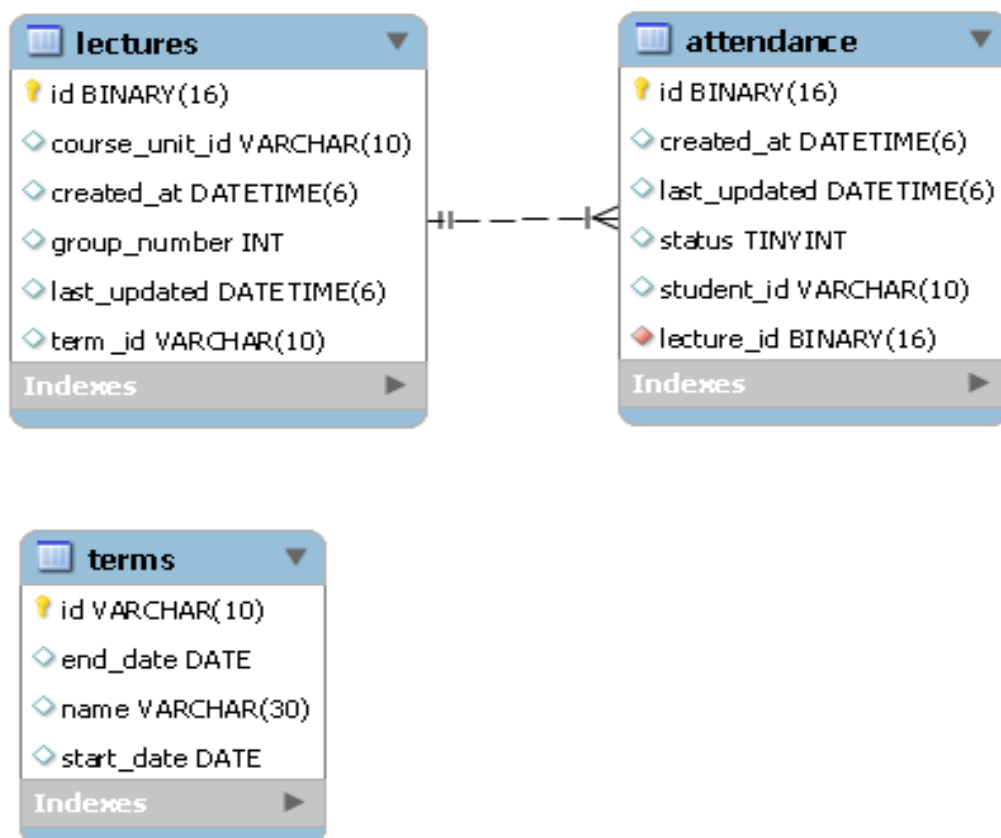
6.7.3 Przeglądanie obecności

Kontroler `AttendanceController` udostępnia dwa endpointy. Jeden z nich przeznaczony jest dla studentów – pozwala im pobrać własną historię obecności dla wybranej grupy zajęciowej. Drugi endpoint dostępny jest wyłącznie dla wykładowców i zwraca zbiorczą mapę obecności wszystkich studentów w danej grupie, gdzie kluczem jest identyfikator studenta, a wartością lista jego obecności.

6.7.4 Aktualny semestr

Mechanizm pobierania aktywnego semestru jest analogiczny do zastosowanego w serwisie grup – `TermService` stosuje cache’owanie w pamięci operacyjnej i pobiera dane z serwisu grup za pośrednictwem klienta Feign w przypadku braku aktualnych danych w bazie.

6.7.5 Schemat bazy danych



Rysunek 8: Schemat bazy danych serwisu obecności

Źródło: opracowanie własne z użyciem MySQL Workbench

Schemat (8) przedstawia trzy tabele:

- **lectures** – przechowuje dane list obecności,
- **attendance** – przechowuje rekordy obecności poszczególnych studentów,
- **terms** – przechowuje dane semestrów analogicznie do serwisu grup;

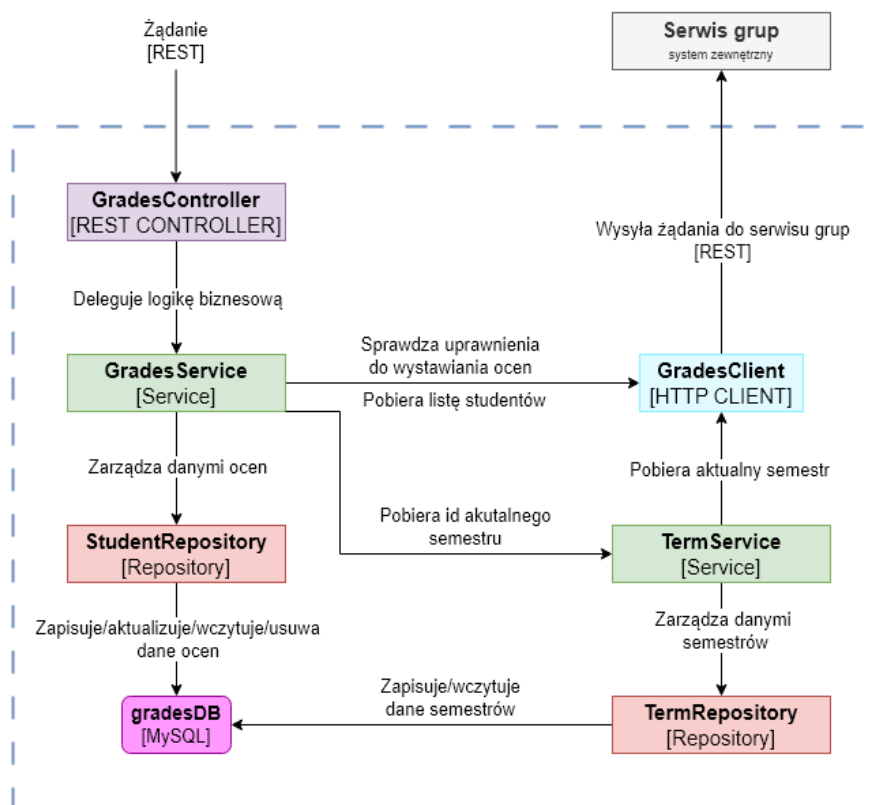
W tabeli **lectures**, **course_unit_id** i **group_number** wiążą listy obecności z grupami zajęciowymi, w których były wystawione. Listy obecności nie są przypisane do wykładowcy, ponieważ ten może zostać zmieniony w toku semestru akademickiego.

Relacja między tabelami **lectures** i **attendance** jest typu jeden-do-wielu. Tabela **terms** nie posiada relacji z tabelą **lectures** – powiązanie z semestrem realizowane jest przez przechowywanie jego identyfikatora w kolumnie **term_id**.

6.8 Serwis ocen

6.8.1 Diagram komponentów

Diagram komponentów serwisu - Serwis ocen



Rysunek 9: Diagram komponentów serwisu ocen

Źródło: opracowanie własne z użyciem draw.io

6.8.2 Zarządzanie ocenami

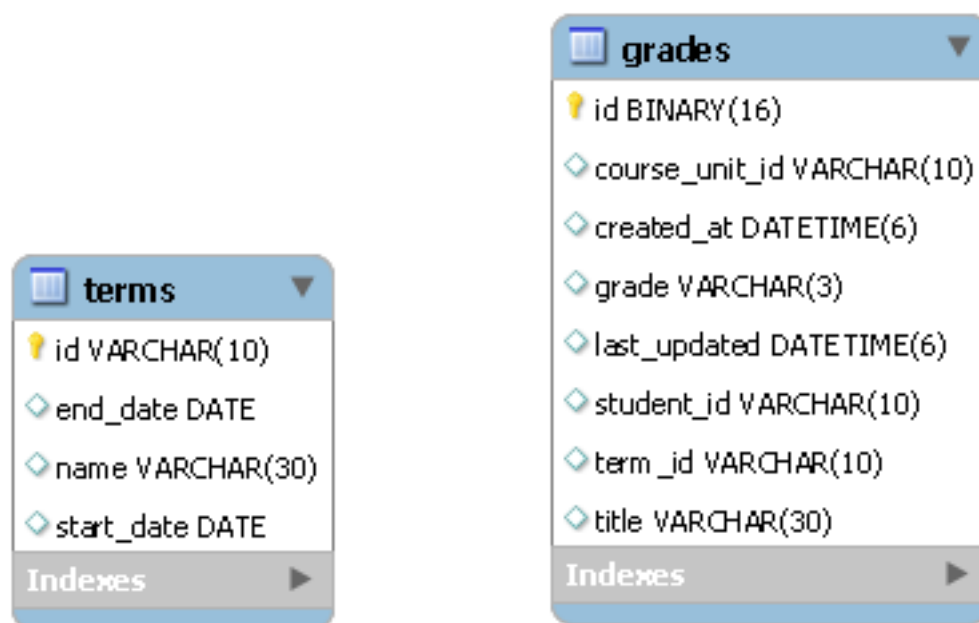
Kontroler `GradesController` udostępnia endpointy umożliwiające zarządzanie ocenami. Każda operacja z nimi związana wiąże się z weryfikacją uprawnień użytkownika, za pomocą mechanizmów bliźniaczych do tych, które zastosowano w przypadku serwisu obecności. Tożsamy jest również sposób w jaki pobierany i przechowywany jest aktualny semestr. System obsługuje nie tylko oceny w tradycyjnej skali, lecz również plusy i minusy – są one identyfikowane jako wartości specjalne 0.1 oraz 0.01.

Kontroler oferuje trzy endpointy służące do wystawiania ocen. Pierwszy z nich, czyli `/api/add` umożliwia wystawienie pojedynczej oceny dla studenta, przy pomocy jego identyfikatora. Endpoint `/api/add/list` umożliwia wystawienie ocen wielu studentom należącym do danej grupy zajęciowej za pomocą jednej operacji. Punkt końcowy `/api/add/exam` dedykowany jest integracji z serwisem sprawdzianów. W jego przypadku student identyfikowany jest za pomocą numeru indeksu.

6.8.3 Przeglądanie ocen

Udostępnione zostały trzy endpointy służące przeglądaniu ocen. Student może pobrać oceny z wybranego przedmiotu lub pięć ostatnio wystawionych ocen. Wykładowca może pobrać zbiorczą mapę ocen wszystkich studentów w danej grupie zajęciowej, gdzie kluczem jest identyfikator studenta, a wartością lista jego ocen.

6.8.4 Schemat bazy danych



Rysunek 10: Schemat bazy danych serwisu ocen

Źródło: opracowanie własne z użyciem MySQL Workbench

Schemat (10) przedstawia dwie tabele:

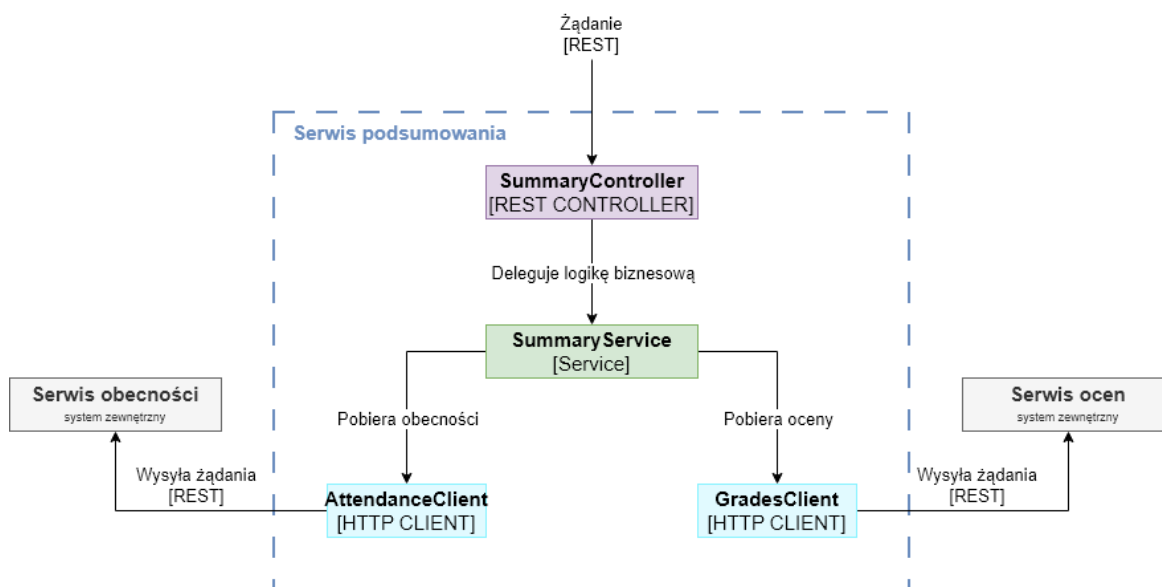
- **grades** – przechowuje dane ocen,
- **terms** – przechowuje dane semestrów;

Tak jak w przypadku serwisu obecności, tabela **terms** posiada jedynie logiczną relację z tabelą **grades**. Pole **course_unit_id** w tabeli ocen wiąże je z odpowiednią jednostką zajęciową kursu akademickiego

6.9 Serwis podsumowania

6.9.1 Diagram komponentów

Diagram komponentów serwisu - Serwis podsumowania



Rysunek 11: Diagram komponentów serwisu podsumowania

Źródło: opracowanie własne z użyciem draw.io

6.9.2 Funkcjonalność

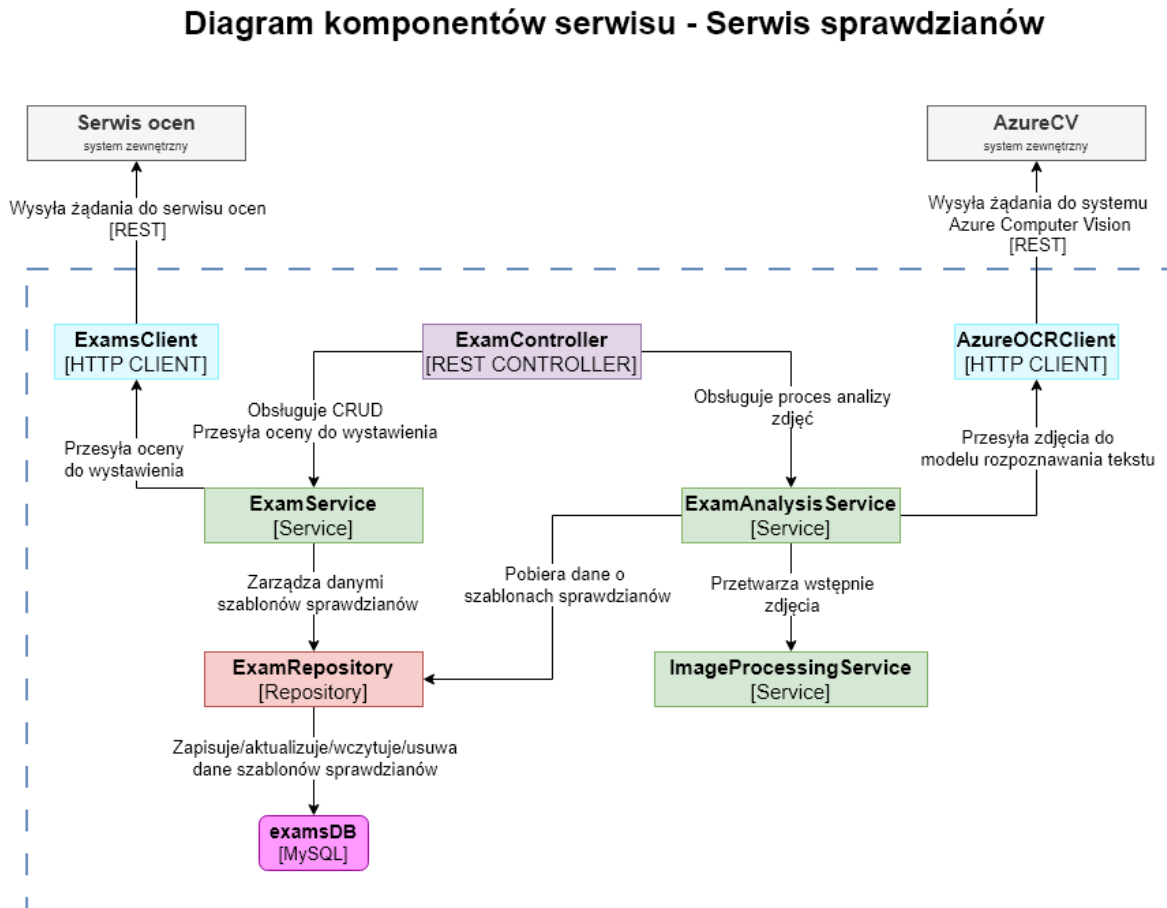
Serwis podsumowania jest najprostszym komponentem systemu – nie przechowuje żadnych danych. Jego jedynym zadaniem jest agregacja odpowiedzi z serwisu ocen oraz serwisu obecności i wygenerowanie podsumowania studenta na ich podstawie.

Kontroler **SummaryController** udostępnia dwa endpointy. Student może pobrać własne podsumowanie dla wybranego przedmiotu, natomiast wykładowca może pobrać zbiorcze podsumowanie wszystkich studentów w danej grupie. **SummaryService** pobiera dane kolejno z obu serwisów za pośrednictwem klientów Feign, a następnie oblicza średnią ocen, liczbę plusów i minusów oraz statystyki obecności – liczbę zajęć na których student był obecny, nieobecny oraz usprawiedliwiony.

6.10 Serwis sprawdzianów

Serwis sprawdzianów jest najbardziej złożonym funkcjonalnie komponentem systemu.

6.10.1 Diagram komponentów



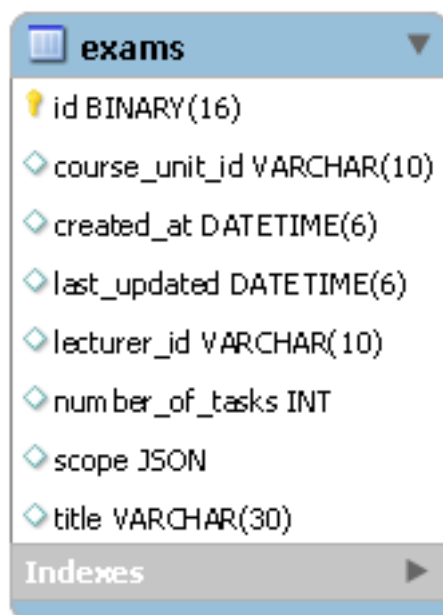
Rysunek 12: Diagram komponentów serwisu sprawdzianów

Źródło: opracowanie własne z użyciem draw.io

6.10.2 Zarządzanie szablonami sprawdzianów

Szablon sprawdzianu przechowuje tytuł, identyfikator jednostki kursowej, liczbę zadań oraz progi punktowe wymagane do uzyskania poszczególnych ocen. Kontroler **ExamController** oferuje endpointy, które pozwalają wykładowcy nimi zarządzać. Realizacja tych operacji CRUD jest delegowana do **ExamService**. Dostęp do szablonu, jak i możliwość zarządzania nim jest zarezerwowana wyłącznie dla wykładowcy, który go utworzył. Przed każdą operacją zachodzi weryfikacja zgodności identyfikatora wykładowcy.

6.10.3 Schemat bazy danych



Rysunek 13: Schemat bazy danych serwisu sprawdzianów

Źródło: opracowanie własne z użyciem MySQL Workbench

Schemat (13) przedstawia pojedynczą tabelę `exams` przechowującą szablony sprawdzianów. Kolumna `scope` przechowuje progi punktowe w formacie JSON – jest to mapa gdzie kluczem jest minimalny wynik procentowy, a wartością odpowiadająca mu ocena. Identyfikator wykładowcy przechowywany jest bezpośrednio w tabeli, bez fizycznej relacji z serwisem autoryzacji, zgodnie z przyjętą w projekcie zasadą relacji logicznych między serwisami.

6.10.4 Przebieg procesu automatycznego oceniania sprawdzianów

Proces automatycznego oceniania sprawdzianów przebiega w kilku etapach. Na początku, wykładowca tworzy szablon sprawdzianu. Na jego podstawie generowana tabela punktacji przeznaczona do dołączenia do arkusza przed wydrukiem. Po przeprowadzeniu sprawdzianu wykładowca przesyła zdjęcie wypełnionej tabeli do serwisu, gdzie jest ono wstępnie przetwarzane, a następnie analizowane przez usługę OCR Azure Computer Vision. Na podstawie zwróconego przez nią tekstu, algorytm identyfikuje numer studenta oraz punktację poszczególnych zadań. Następnie oblicza wynik i wyznacza ocenę na podstawie progów procentowych z użytego szablonu. Po zatwierdzeniu wyniku analizy przez wykładowcę, ocena wystawiana jest za pośrednictwem serwisu ocen.

6.10.5 Specyfika wykorzystanego modelu OCR

Azure Computer Vision udostępnia model OCR oparty na głębokich sieciach neuronowych, zoptymalizowanych przede wszystkim pod kątem tekstu maszynowego. W kontekście projektu zidentyfikowano kilka ograniczeń modelu istotnych dla procesu analizy

sprawdzianów. Model nie radzi sobie dobrze z rozpoznawaniem pisma odręcznego na obróconych zdjęciach, z czego wynika konieczność uprzedniej korekcji orientacji obrazu. Wykazuje on również trudności w rozpoznawaniu czerwonego atramentu, który charakteryzuje się niskim kontrastem względem białego tła w przestrzeni kolorów wykorzystywanej przez model. Problem ten rozwiązano przez wzmocnienie czerwonych pikseli w procesie przetwarzania wstępnego. Dodatkowo zdarza się mu mylić podobnie wizualne znaki – przykładem jest mylenie litery B z cyfrą 8, co zostało uwzględnione w algorytmie analizy jego odpowiedzi poprzez normalizację rozpoznanych znaków przed ich przetworzeniem. Ostatnim istotnym ograniczeniem jest to, że przerwy w atramencie mogą powodować nieprawidłowe rozpoznanie znaków.

Pomimo opisanych ograniczeń, model OCR okazał się wystarczająco solidny do zastosowania w projekcie – potrafi rozpoznać pismo odręczne nawet na zdjęciach o niskiej jakości.

6.10.6 Format punktacji

Biorąc pod uwagę specyfikę wykorzystanego modelu OCR zdecydowano się na wykorzystanie liter A i B jako symboli punktacji – są to znaki które model rozpoznaje z wysoką pewnością i rzadko myli z innymi symbolami. Litera A oznacza jeden punkt, a litera B pół punktu. Brak punktacji lub jakikolwiek inny znak oznacza 0 punktów.

6.10.7 Generacja tabeli punktacji

Tabela punktacji generowana jest programistycznie przy użyciu biblioteki **Graphics2D** wchodzącej w skład standardowej biblioteki Java. Na podstawie liczby zadań zdefiniowanej w szablonie sprawdzianu obliczane są wymiary obrazu, a następnie rysowane są kolejne wiersze tabeli – każdy wiersz składa się z pola z numerem zadania oraz pustego pola przeznaczonego na wpisanie odpowiedzi. Nad tabelą umieszczone jest pole na numer indeksu studenta w postaci podkreśleń.

1)	
2)	
3)	
4)	
5)	

Rysunek 14: Przykład wygenerowanej tabeli punktacji dla 5 zadań

Źródło: wygenerowane przez aplikację webowy dziennik ocen studenta

Na rysunku (14) można zauważyć, że powyżej tabeli wyrenderowane są dwa znaczniki orientacyjne – trójkąt "▲" oraz kwadrat "■". Wykorzystywane są one na etapie

wstępnego przetwarzania obrazu do określenia orientacji zdjęcia. Wygenerowany obraz zwracany jest w formacie JPG.

6.10.8 Wstępne przetwarzanie obrazu

Przed wysłaniem zdjęcia do Azure Computer Vision, `ImageProcessingService` przetwarza je wstępnie przy użyciu biblioteki OpenCV. Przetwarzanie obejmuje dwa etapy. Pierwszy z nich polega na wzmocnieniu czerwonego atramentu – obraz konwertowany jest do przestrzeni barw HSV²², a piksele w zakresie czerwieni są przyciemniane w celu poprawy kontrastu. Drugi etap koryguje orientację obrazu – algorytm wykrywa dwa znaczniki orientacyjne wydrukowane na szablonie: trójkąt równoboczny oraz kwadrat. Na podstawie ich wzajemnego położenia obliczany jest kąt obrotu i obraz jest prostowany.

6.10.9 Analiza tabeli punktacji

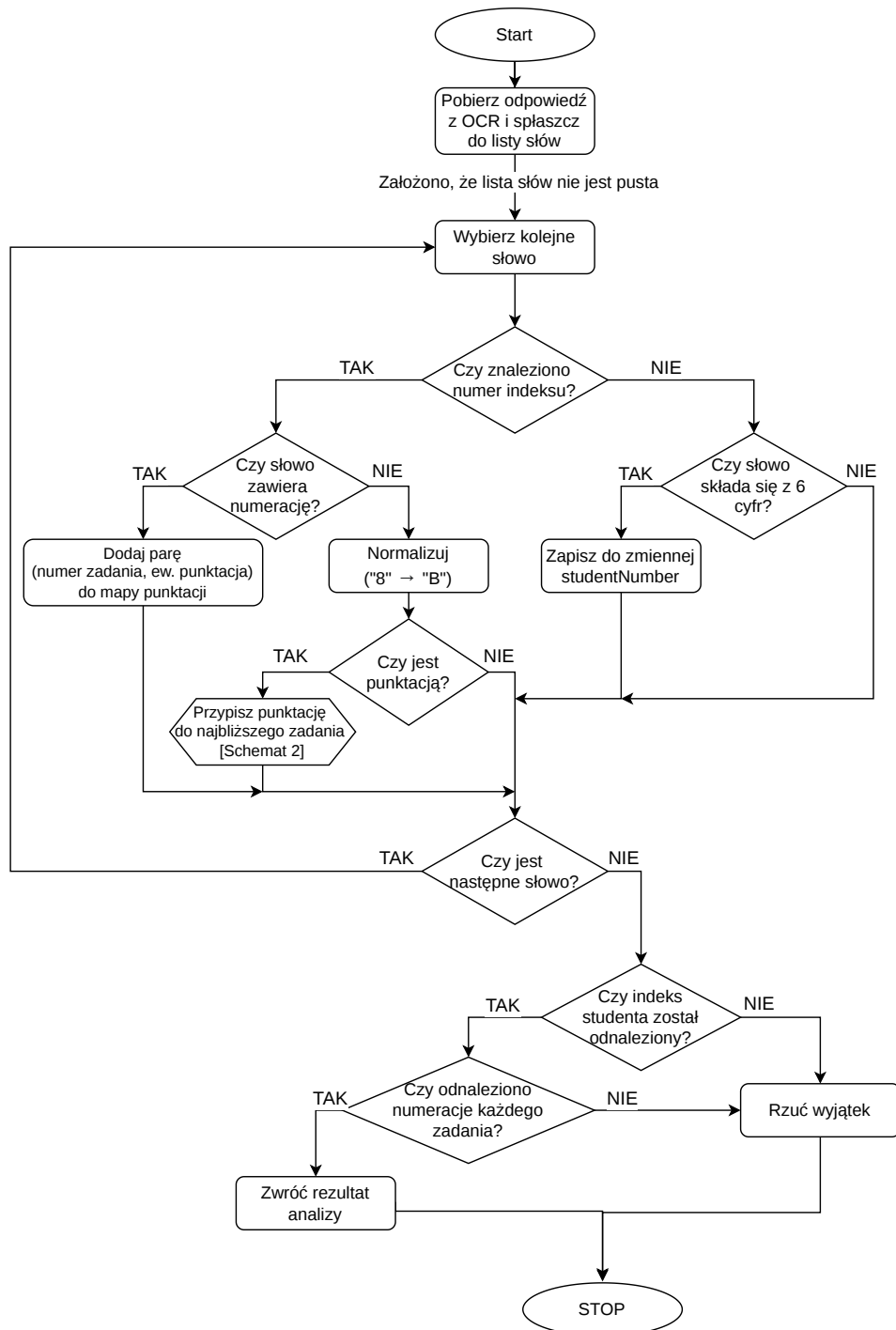
Przetworzone zdjęcie jest wysłane do Azure Computer Vision przez `AzureOCRClient`. Odpowiedź zawiera listę rozpoznanych słów wraz z ich współrzędnymi na obrazie. Komponent `ExamAnalysisService` przekazuje odpowiedź do algorytmu analizy, który identyfikuje numer indeksu studenta, przypisuje odpowiedzi do zadań i oblicza wynik. Wynikiem analizy jest obiekt `ScanResultDto`, który zawiera następujące pola: liczbę zadań, mapę punktacji poszczególnych zadań, łączny wynik punktowy, wynik procentowy, wyznaczoną ocenę oraz numer indeksu studenta.

6.10.10 Działanie algorytmu analizy tabeli punktacji

Algorytm analizy tabeli punktacji przetwarza listę słów zwróconą przez model rozpoznawania tekstu z usługi Azure Computer Vision. Jego działanie zostało przedstawione za pomocą schematu blokowego na kolejnych stronach. Schemat 1 (15) przedstawia główną pętlę algorytmu, schemat 2 (16) przedstawia procedurę przypisywania punktacji do zadania.

²²HSV (ang. *Hue*, *Saturation*, *Value*, pol. barwa, nasycenie, jasność) – model reprezentacji kolorów, w którym kolor opisywany jest przez typ barwy, jej intensywność oraz jasność, co ułatwia filtrowanie kolorów w porównaniu do modelu RGB.

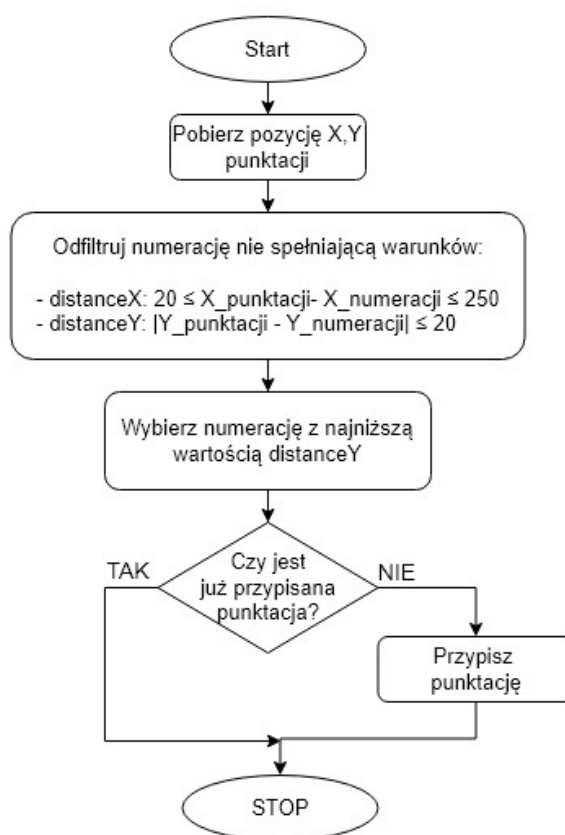
Schemat działania algorytmu analizy tabeli punktacji [Schemat 1]



Rysunek 15: Schemat algorytmu analizy tabeli punktacji

Źródło: opracowanie własne z użyciem draw.io

Schemat działania procedury przypisania punktacji do zadania [Schemat 2]



Rysunek 16: Schemat procedury przypisywania punktacji do zadania

Źródło: opracowanie własne z użyciem draw.io

W pierwszej fazie algorytm wyszukuje numer indeksu studenta – każde kolejne słowo jest sprawdzane pod kątem zgodności ze wzorcem sześciu cyfr. Słowa nie spełniające tego wzorca są pomijane do momentu odnalezienia numeru indeksu. Przyjęto taką strategię ze względu kolejność słów zwróconych z OCR – od góry do dołu.

W drugiej fazie algorytm przetwarza kolejne słowa w poszukiwaniu numeracji zadań oraz odpowiadających im punktacji. Słowa pasujące do odpowiedniego wzorca regex traktowane są jako numer zadania – tworzony jest kandydat na odpowiedź przechowujący pozycję X,Y słowa na obrazie oraz ewentualną punktację, jeśli odpowiedź znajduje się w tym samym słowie co numeracja. Przed sprawdzeniem czy słowo jest punktacją, przeprowadzana jest normalizacja. Znak 8 zamieniany jest na B, aby skompensować błędy modelu OCR. Punktacja znajdująca się w osobnym słowie przypisywana jest do zadania zgodnie z procedurą opisaną w schemacie 2 (16).

Procedura przypisywania punktacji do zadania filtruje kandydatów na podstawie ich pozycji względem numeracji. Odległość pozioma musi mieścić się w przedziale $\langle 20, 250 \rangle$ pikseli, a wartość bezwzględna odległości pionowej nie może być większa niż 20 pikseli. Spośród kandydatów spełniających te warunki wybierany jest ten o najmniejszej odległości pionowej. Punktacja przypisywana jest tylko jeśli kandydat nie ma jeszcze przypisanej wartości.

Po przetworzeniu wszystkich słów algorytm weryfikuje czy numer indeksu został odnaleziony oraz czy wszystkie zadania mają przypisane odpowiedzi. W przypadku niespełnienia któregoś z tych warunków rzucany jest wyjątek. W przeciwnym razie obliczany jest wynik procentowy i wyznaczana jest ocena na podstawie progów zdefiniowanych w szablonie sprawdzianu.

6.10.11 Analiza złożoności algorytmu

Na potrzeby analizy przyjęto następujące oznaczenia:

- n – liczba słów zwróconych przez usługę Azure Computer Vision OCR,
- q – liczba zadań w sprawdzianie;

Pierwszym etapem jest spłaszczenie struktury odpowiedzi OCR do listy pojedynczych słów przy użyciu operacji `flatMap`. Wymaga ona jednokrotnego przejścia przez wszystkie elementy struktury, co daje złożoność $O(n)$.

Drugim etapem jest główna pętla przetwarzania słów. Iteruje ona po każdym słowie z listy, wykonując dla każdego z nich serię operacji warunkowych. W przypadku, gdy dane słowo zostanie zidentyfikowane jako punktacja, wywołana jest procedura przydzielenia jej do najbliższego zadania. Przeszukuje ona liniowo wartości mapy kandydatów `answers`, zawierającą co najwyżej q wpisów. Złożoność tego etapu wynosi zatem $O(n * q)$.

W trzecim etapie zachodzi walidacja kompletności znalezionej numeracji. Procedura walidacji wykonuje iterację po liczbach całkowitych od 1 do q , weryfikując obecność każdego pytania w mapie kandydatów. Dostęp do struktury `HashMap` jest operacją o średniej złożoności $O(1)$, co daje złożoność etapu $O(q)$.

Czwarty etap to obliczenie wyniku końcowego. Metoda `computeResult` iteruje po wszystkich q numeracjach. Mapa progów `scope` jest przechowywana w bazie danych w postaci zserializowanego obiektu `TreeMap`, więc przy odczycie encji jest ona dostępna jako odpowiednia struktura. Operacje wstawiania wyników do mapy wynikowej `result`, która również jest typu `TreeMap`, kosztują $O(\log q)$ każda. Złożoność etapu czwartego wynosi zatem $O(q \log q)$.

Etapem piątym jest wyznaczenie oceny końcowej. Metoda przeszukuje liniowo mapę progów `scope`, która zgodnie z założeniami dziedzinowymi systemu zawsze zawiera dokładnie pięć progów procentowych. Złożoność tego etapu jest zatem stała i wynosi $O(1)$.

Dominującym składnikiem złożoności algorytmu jest etap przetwarzania słów. Złożoność czasowa wynosi $O(n * q)$.

Złożoność pamięciowa wynosi $O(n + q)$, gdzie $O(n)$ odpowiada przechowywaniu listy słów, a $O(q)$ za mapę kandydatów **answers** oraz wynikową mapę **result**.

Mimo, że asymptotyczna złożoność wynosi $O(n * q)$, w warunkach rzeczywistych wartość q jest ograniczona przez liczbę zadań sprawdzianu i typowo mieści się w przedziale od kilku do kilkudziesięciu. Parametr q można zatem traktować jako pewnego rodzaju stałą, co w praktyce redukuje złożoność czasową do $O(n)$.

7 Implementacja frontendu

7.1 Architektura

7.1.1 Podejście modułowe

Kod źródłowy aplikacji zorganizowano według podejścia modułowego – pliki są grupowane według przynależności do danej funkcjonalności, a nie według rodzaju.

Struktura plików katalogu z kodem źródłowym `/src` wygląda następująco:

```
src/
├── features/
│   ├── auth/ .....obsługa uwierzytelniania, zarządzanie sesją, ochrona tras
│   ├── course/ ..... obsługa kursów, ocen i frekwencji
│   ├── exams/ .....obsługa automatycznego oceniania sprawdzianów
├── components/ ..... współdzielone komponenty (pasek nawigacji)
└── pages/ ..... widoki przypisane do tras nawigacji
```

Każdy moduł zawiera własne podkatalogi na komponenty, hooki²³, typy i funkcje pomocnicze. Ich publiczne interfejsy eksportowane są przez pliki `index.ts`. Takie podejście ukrywa szczegóły implementacji i pozwala używać modułu wyłącznie przez zdefiniowane punkty wejścia, co poprawia czytelność kodu i ułatwia wprowadzanie zmian.

7.1.2 Funkcje typu hook jako warstwa logiki

Funkcje typu hook w React to mechanizm pozwalający wydzielić logikę stanową poza komponent i współdzielić ją między wieloma komponentami bez dziedziczenia. Logika pobierania danych i operacje na zasobach zostały zaimplementowane za pomocą wcześniej wspomnianych funkcji, na przykład: `useGrades`, `useAttendance`. Komponenty wizualne są odpowiedzialne wyłącznie za renderowanie. Otrzymują one dane i procedury obsługi zdarzeń przez parametry.

Zarządzanie stanem aplikacji opiera się wyłącznie na wbudowanym mechanizmie kontekstu React oraz standardowych funkcjach `useState` oraz `useEffect`.

7.2 System uwierzytelniania i kontroli dostępu

7.2.1 Proces autoryzacji przez USOS

Proces autoryzacji rozpoczyna się na stronie logowania, która prezentuje użytkownikowi przycisk przenoszący go do okna logowania w systemie USOS. Po pomyślnej weryfikacji wprowadzonych przez niego danych, USOS odsyła użytkownika z powrotem do aplikacji. Backend aplikacji weryfikuje odpowiedź i wystawia token JWT przechowywany w ciasteczku sesji. Następnie przeglądarka trafia na widok `LoginSuccessPage`, który sprawdza sesję i kieruje użytkownika do odpowiedniego widoku w zależności od jego roli.

²³Hooki (*ang. hooks*) – funkcje wprowadzone w React 16.8 umożliwiające korzystanie ze stanu i innych funkcjonalności React w komponentach funkcyjnych, bez konieczności pisania komponentów klasowych. Przykłady: `useState`, `useEffect`, `useContext`.

7.2.2 Zarządzanie sesją

Komponent `AuthProvider` udostępnia przez kontekst React dane zalogowanego użytkownika. Przy pierwszym zamontowaniu komponentu wysyła on żądanie do endpointu `/gradebook/auth/me` w celu uzyskania danych na temat użytkownika i potwierdzenia ważności sesji.

Aby ograniczyć liczbę niepotrzebnych żądań sieciowych podczas przeładowania strony, dane zalogowanego użytkownika są przechowywane w lokalnej pamięci podręcznej przeglądarki `localStorage`. Przy każdym kolejnym ponownym zamontowaniu komponentu `AuthProvider` aplikacja odczytuje dane z pamięci podręcznej i pomija wysłanie żądania do serwera. Czas ważności zapisu w pamięci jest tożsamy z czasem ważności tokenu JWT – trwa dwie godziny. Gdy sesja się przedawni, użytkownik przenoszony jest ponownie na stronę logowania.

7.2.3 Kontrola dostępu oparta na rolach

Aplikacja wyróżnia dwie role użytkownika: student i wykładowca. Ochrona dostępu realizowana jest na dwóch poziomach.

Na poziomie nawigacji zastosowano komponenty opakowujące poszczególne trasy:

- `ProtectedRoute` – weryfikuje uwierzytelnianie, niezalogowany użytkownik jest przekierowywany na stronę logowania,
- `StudentRoute` i `LecturerRoute` – weryfikują uwierzytelnianie oraz rolę, użytkownik z nieodpowiednią rolą trafia na stronę informującą o braku dostępu;

Na poziomie komponentów zastosowano warunkowe renderowanie za pośrednictwem komponentów `StudentOnly` i `LecturerOnly`, które determinują rendering fragmentów interfejsu dla danej roli.

7.3 Moduły funkcjonalne

7.3.1 Moduł kursów

Moduł kursów obsługuje dwa odrębne widoki – widok wykładowcy `CourseManagePage` służący do zarządzania grupą oraz widok studenta `CoursePerformancePage` umożliwiający podgląd własnych wyników pracy na zajęciach. Oba widoki korzystają ze wspólnego komponentu zakładek z trzema sekcjami.

Pierwsza sekcja obsługuje oceny. Wykładowca może dodawać oceny jednostkowo lub zbiorowo, a także edytować i usuwać istniejące wpisy. Student może zobaczyć uzyskane przez siebie oceny.

Kolejna jest odpowiedzialna za obecność. Wykładowca tworzy listy obecności oznaczając obecność studentów w trzech kategoriach: obecny, nieobecny i usprawiedliwiony. Może także edytować i usuwać wpisy związane z frekwencją. Student widzi tylko i wyłącznie stan swojej frekwencji.

Ostatnia obsługuje podsumowanie. Wykładowca może zobaczyć zestawienie statystyczne wyników pracy na zajęciach dla wszystkich studentów z jego grupy. Student może zobaczyć tylko podsumowanie swoich wyników.

Operacje modyfikujące dane realizowane są za pośrednictwem okien dialogowych. Stanem tych okien zarządzają hooki `useGrades` oraz `useAttendance`, które udostępniają komponentom dane oraz procedury obsługi zdarzeń.

7.3.2 Moduł sprawdzianów

Moduł sprawdzianów przeznaczony jest wyłącznie dla wykładowców. Umożliwia tworzenie i edycję szablonów sprawdzianów wraz z konfigurowalnymi progami ocen. Dla każdego szablonu dostępne są dwie operacje charakterystyczne dla systemu.

Pierwszą z nich jest wygenerowanie tabeli punktacji. Żądanie wysłane do serwera zwraca plik w formacie JPG, który aplikacja udostępnia do pobrania.

Drugą jest automatyczne ocenianie. Po przesłaniu zdjęcia tabeli punktacji po stronie backendu wykonywana jest analiza. Po jej zakończeniu, wykładowca musi zatwierdzić jej wynik, który może także uprzednio edytować. Jego oczom ukaże się wtedy informacja, o tym czy wystawienie oceny przebiegło pomyślnie.

7.4 Komunikacja z backendem

Frontend komunikuje się z backendem za pomocą żądań HTTP do jego REST API. Do wysyłania żądań stosowany jest wbudowany mechanizm przeglądarki – funkcja `fetch`. Każde żądanie zawiera parametr `credentials: "include"`, dzięki czemu ciasteczko zawierające JWT jest automatycznie do nich dołączane.

8 Testowanie

8.1 Testy jednostkowe

Każdy mikroservis posiada zestaw testów jednostkowych zrealizowanych przy pomocy bibliotek JUnit 5 oraz Mockito. Testy obejmują warstwę **service**. Zależności zewnętrzne takie jak repozytoria, czy klienci Feign są zastępowane obiektami zastępczymi (ang. *mock*). Pozwala to na izolowane testowanie logiki biznesowej bez konieczności uruchamiania całej infrastruktury. Do testowania serwisów korzystających z bazy danych wykorzystano bazę H2 działającą w pamięci operacyjnej.

8.2 Testy manualne

Funkcjonalność systemu była weryfikowana w sposób manualny ze względu na specyfikę przyjętej architektury oraz integrację z systemami zewnętrznymi. Każdy endpoint był testowany bezpośrednio po implementacji przy użyciu narzędzia Postman. Frontend był testowany poprzez interakcję z interfejsem użytkownika w przeglądarce internetowej.

8.3 Testowanie systemu analizy zdjęć tabeli punktacji

Proces został przetestowany na zbiorze 12 zdjęć pochodzących z prawdziwego sprawdzianu[14]. Spośród nich 9 zostało przeanalizowanych całkowicie poprawnie, co odpowiada skuteczności 75 procent. Wszystkie niepoprawne analizy wynikały z ograniczeń modelu OCR lub nieczytelnego pisma, a nie z błędów w implementacji algorytmu. Żaden z przypadków niepowodzenia nie był spowodowany wadą logiki przypisywania punktacji do zadań.

Przyczyny niepowodzeń analizy we wspomnianych przypadkach:

- nieodczytany indeks – w tym przypadku był napisany bardzo niewyraźnie, a zdjęcie było rozmazane,
- nieodczytanie całego rzędu w tabeli – przypadek niedokładności modelu OCR,
- przerwy w tuszu i odczytanie zamazanej punktacji – ograniczenia modelu oraz przypadek jego nadmiernej czułości;

9 Wdrożenie systemu

9.1 Środowisko wdrożeniowe

System został wdrożony przy użyciu narzędzia Docker Compose, które umożliwia deklaratywne definiowanie wielokontenerowych aplikacji. Całość infrastruktury opisana jest w dwóch plikach konfiguracyjnych: `docker-compose.yml`, zawierającym definicje poszczególnych serwisów, oraz `common-config.yml`, przechowującym konfigurację współdzieloną między serwisami. Zastosowane podejście pozwala na uruchomienie całego systemu w jednym kroku, za pomocą polecenia `docker compose up`. Równie łatwe jest jego zatrzymanie, wykonywane za pomocą polecenia `docker compose down`.

9.2 Architektura kontenerów

System składa się z następujących kontenerów:

- bazy danych – pięć izolowanych instancji MySQL, każda dedykowana jednemu serwisowi,
- mikroserwisy – siedem serwisów Spring Boot, w tym API Gateway,
- pamięć podręczna – magazyn Redis,
- warstwa wejściowa – nginx pełniący rolę odwrotnego serwera proxy²⁴,
- frontend – kontener z aplikacją React serwowaną przez swoją wewnętrzną instancję nginx;

9.3 Sieć i izolacja

Wszystkie kontenery należą do zdefiniowanej sieci mostkowej²⁵ o nazwie `gradebook`. Komunikacja między kontenerami odbywa się wyłącznie wewnątrz tej sieci, przy użyciu nazw kontenerów jako hostów DNS²⁶.

Żadna baza danych, mikroserwis ani cache nie posiada portów wystawionych na interfejs hosta. Jedynym punktem wejścia do systemu z zewnątrz jest kontener nginx, który nasłuchuje na portach 80 i 443.

²⁴Odwrotny serwer proxy (ang. *reverse proxy*) – serwer pośredniczący w komunikacji między klientem a serwerami docelowymi, przyjmujący zapytania w imieniu tych serwerów i przekazujący je do odpowiednich komponentów systemu.

²⁵Sieć mostkowa (ang. *bridge network*) – wirtualna sieć tworzona przez Docker, umożliwiająca izolowaną komunikację między kontenerami przy użyciu wewnętrznych adresów IP i nazw hostów.

²⁶DNS (ang. *Domain Name System*, pol. system nazw domenowych) – system tłumaczący nazwy hostów na adresy IP, umożliwiający komunikację sieciową przy użyciu czytelnych nazw zamiast adresów numerycznych.

9.4 Szyfrowanie komunikacji

9.4.1 TLS na warstwie HTTP

Nginx terminuje połączenia TLS²⁷ na porcie 443. Certyfikat SSL²⁸ oraz klucz prywatny montowane są do kontenera jako wolumen ze ścieżki `./nginx/certs`. Ruch przychodzący na port 80 (HTTP) jest automatycznie przekierowywany na HTTPS z kodem odpowiedzi 301.

Nginx rozróżnia ruch na podstawie ścieżki URL – żądania pod prefiksem `/gradebook` kierowane są do API Gateway, pozostałe żądania natomiast trafiają do frontendu.

9.4.2 SSL w połączeniach z bazami danych

Połączenia pomiędzy mikroserwisami a bazami danych wymagają szyfrowania TLS poprzez parametr `sslMode=REQUIRED` w URL połączenia.

9.5 Trwałość danych

Dane wszystkich baz danych przechowywane są w nazwanych wolumenach Dockera, co zapewnia ich przetrwanie między restartami kontenerów.

9.6 Kolejność uruchamiania i kontrole stanu

Docker Compose zarządza kolejnością uruchamiania kontenerów na podstawie zależności `depends_on` z warunkiem `service_healthy`. Każdy serwis posiada zdefiniowaną kontrolę stanu (*ang. health check*). Bazy danych sprawdzane są przez komendę `mysqladmin ping`, mikroserwisy przez endpoint `/actuator/health/readiness` wystawiony przez Spring Boot Actuator. Każdy mikroserwis przed rozpoczęciem działania oczekuje na sygnały o gotowości serwisów, od których jest zależny. Nginx startuje ostatni, dopiero po pozytywnym rezultacie kontroli stanu API Gateway.

9.7 Zarządzanie sekretami

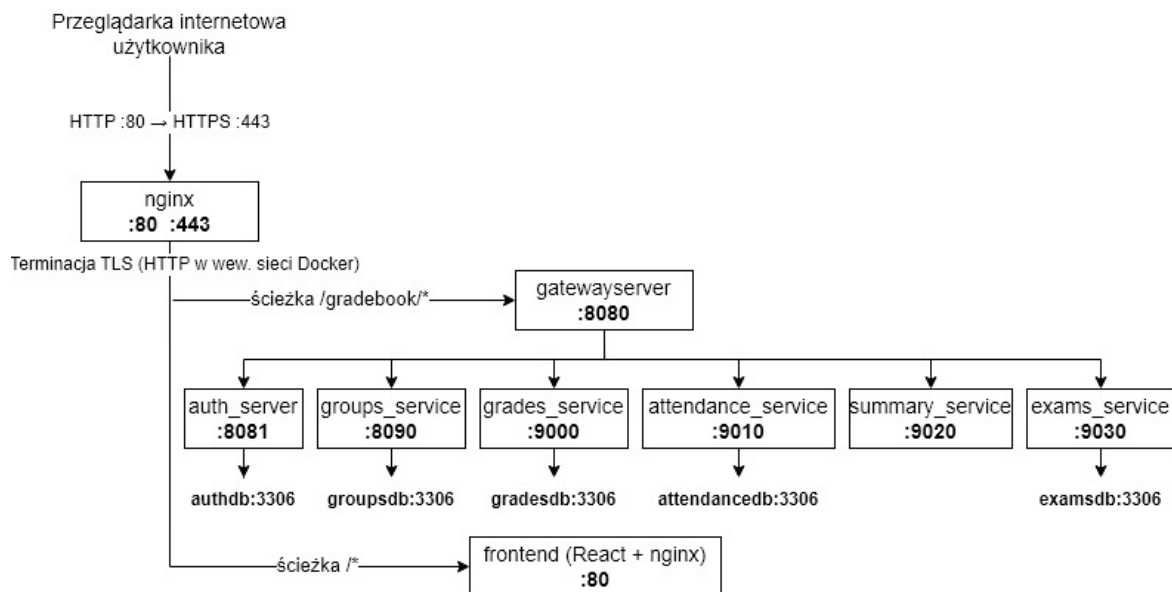
Wrażliwe dane konfiguracyjne są przechowywane w pliku `.env`, który nie powinien nigdy być umieszczany w repozytorium kodu źródłowego. Docker Compose automatycznie wczytuje ten plik oraz podstawia zmienne środowiskowe w miejscach oznaczonych składnią `$NAZWA_ZMIENNEJ`.

²⁷Terminacja TLS (*ang. TLS termination*) – proces odszyfrowywania ruchu HTTPS przez pośredni serwer, który następnie przekazuje żądania do wewnętrznej sieci jako niezaszyfrowany HTTP.

²⁸Certyfikat SSL/TLS – cyfrowy dokument potwierdzający tożsamość serwera oraz zawierający klucz publiczny wykorzystywany do nawiązania szyfrowanego połączenia.

9.8 Przepływ żądania

Poniżej przedstawiony został schemat przepływu żądania przez infrastrukturę systemu.



Rysunek 17: Schemat przepływu żądania

Źródło: opracowanie własne z użyciem draw.io

Jak przedstawiono na schemacie, żądanie wysłane przez przeglądarkę użytkownika trafia najpierw do kontenera nginx, który terminuje połączenie TLS i przekierowuje ruch do właściwego komponentu na podstawie ścieżki URL.

10 Podsumowanie

10.1 Napotkane problemy

10.1.1 Architektura mikroserwisowa

Projekt był pierwszym doświadczeniem autora z architekturą mikroserwisową. Wiązało się to z koniecznością poznania nowych wzorców projektowych oraz narzędzi. Szczególnym wyzwaniem było zaprojektowanie komunikacji między serwisami oraz propagacja kontekstu bezpieczeństwa, czyli JWT.

10.1.2 Integracja z USOS API

Największym wyzwaniem przy integracji z USOS API było pobranie numerów indeksu studentów. Pierwotnie zakładano, że będzie on dostępny bezpośrednio z poziomu metody pobierającej grupy zajęciowe. Okazało się jednak, że USOS API nie zwraca tej informacji w ramach tego endpointu. Konieczne okazało się wysłanie osobnego żądania dla każdego studenta z osobna. Rozwiązanie jest mniej wydajne niż zakładano, jednak dzieje się to rzadko – tylko jeśli student jeszcze nie ma rekordu w bazie danych.

10.1.3 Moduł automatycznego oceniania sprawdzianów

Pierwszym wyzwaniem w procesie implementacji modułu analizy sprawdzianów był wybór odpowiedniego silnika OCR. Wstępne próby z całkowicie darmowym API wykazały zbyt niską skuteczność rozpoznawania pisma ze zdjęć. Ostatecznie wybrano usługę Azure Computer Vision, która oferuje znacznie wyższą skuteczność. Warto zaznaczyć, że usługa ta dostępna jest w bezpłatnym planie pozwalającym na 5000 wywołań miesięcznie, co w praktyce pozwala na przeanalizowanie 2500 zdjęć, ponieważ każda analiza wymaga dwóch wywołań API. Limit ten jest w pełni wystarczający na potrzeby projektu akademickiego.

Kolejnym problemem były ograniczenia modelu OCR opisane szczegółowo w rozdziale szóstym. Został on rozwiązany z pomocą biblioteki OpenCV na etapie wstępnego przetwarzania.

10.2 Wnioski końcowe

Aplikacja webowa wykonana w ramach niniejszej pracy spełniła założone cele. Umożliwia wykładowcom zarządzanie ocenami częściowymi i obecnością studentów w ramach prowadzonych grup zajęciowych. Udostępnia też studentom wgląd do ich wyników. Automatyzuje także proces oceniania kolokwii z wykorzystaniem technologii OCR.

Realizacja projektu pozwoliła na zdobycie praktycznego doświadczenia w projektowaniu i implementacji systemów opartych na architekturze mikroserwisowej. Zastosowanie tej architektury okazało się uzasadnione – podział odpowiedzialności między mikroserwisy ułatwia utrzymanie i rozwój systemu.

Z powodu podjętej tematyki, konieczne było uwzględnienie wymagań RODO dotyczących ochrony danych osobowych studentów. Zakres przechowywanych danych ograni-

czono do minimum. Wykorzystano restrykcyjną strategię w przypadku przechowywania tokenu dostępu do USOS użytkownika – jest szyfrowany i przechowywany do końca dwu godzinnej sesji lub momentu wylogowania. Komunikacja z systemem odbywa się wyłącznie przez szyfrowane połączenie HTTPS. Dostęp do danych jest kontrolowany przez mechanizm oparty na rolach użytkownika oraz weryfikację uprawnień w konkretnych grupach zajęciowych.

Moduł automatycznego oceniania sprawdzianów, pomimo ograniczeń wynikających z charakterystyki modelu OCR, okazał się skuteczny w praktycznym zastosowaniu. Skuteczność na poziomie 75% w testach na rzeczywistych sprawdzianach jest wynikiem zadowalającym.

10.3 Możliwości rozwoju

Architektura projektu pozwala na łatwe dodawanie nowych modułów lub rozwój już istniejących. Możliwości rozwoju jest bardzo dużo, jednak przed dodaniem czegokolwiek warto się zastanowić czy dana funkcjonalność nie jest już dostarczana prowadzącym przez USOS czy inne wykorzystywane systemy.

Przykładowym ulepszeniem mogłaby być aplikacja mobilna ułatwiająca sprawdzanie obecności. Studenci przykładając swoją legitymację studencką do telefonu prowadzącego mogliby sami zaznaczać swoją obecność na zajęciach. Taka funkcjonalność może być zrealizowana dzięki temu, że legitymacje te są kartami zbliżeniowymi NFC²⁹. Chip w nich zawarty przechowuje informacje na temat studenta, które aplikacja mogłaby czytać i przesyłać do API w celu zaznaczenia obecności.

Kolejnym możliwym kierunkiem rozwoju jest udoskonalenie modułu automatycznego oceniania sprawdzianów. W aktualnym stanie, generowana tabela punktacji może mieć tylko dwie kolumny, co może być uciążliwe w przypadku dużych sprawdzianów. Dodatkowo, system przewiduje tylko trzy opcje punktowania zadania – zero, pół lub jeden punkt. Tu także jest miejsce na ulepszenia.

²⁹NFC (ang. *Near Field Communication*, pol. komunikacja bliskiego zasięgu) – standard bezprzewodowej komunikacji krótkiego zasięgu działający na częstotliwości 13,56 MHz, umożliwiający wymianę danych między urządzeniami znajdującymi się w odległości do kilku centymetrów.

Bibliografia

- [1] Rozporządzenie parlamentu europejskiego i rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych i w sprawie swobodnego przepływu takich danych oraz uchylenia dyrektywy 95/46/WE (ogólne rozporządzenie o ochronie danych). Dziennik Urzędowy Unii Europejskiej L 119 z 4.5.2016, s. 1–88, 2016.
- [2] Sejm Rzeczypospolitej Polskiej. Ustawa z dnia 20 lipca 2018 r. – prawo o szkolnictwie wyższym i nauce. Dz.U. 2018 poz. 1668, 2018. z późn. zm.
- [3] C4 Model. The C4 model for visualising software architecture.
<https://c4model.com/>
[dostęp: 02.03.2026].
- [4] Martin Fowler. Repository.
<https://martinfowler.com/eaCatalog/repository.html>
[dostęp: 02.03.2026].
- [5] Martin Fowler. Data transfer object.
<https://martinfowler.com/eaCatalog/dataTransferObject.html>
[dostęp: 02.03.2026].
- [6] Martin Fowler. Inversion of control containers and the dependency injection pattern.
<https://martinfowler.com/articles/injection.html>
[dostęp: 02.03.2026].
- [7] Refactoring Guru. Singleton.
<https://refactoring.guru/design-patterns/singleton>
[dostęp: 02.03.2026].
- [8] Martin Fowler. Circuit breaker.
<https://martinfowler.com/bliki/CircuitBreaker.html>
[dostęp: 02.03.2026].
- [9] CyberArk. What is rsa encryption?
<https://www.cyberark.com/what-is/rsa-encryption/>
[dostęp: 02.03.2026].
- [10] Uniwersytecki System Obsługi Studiów. Autoryzacja – usos api.
<https://apps.usos.edu.pl/developers/api/authorization/>
[dostęp: 02.03.2026].
- [11] National Institute of Standards and Technology. Advanced encryption standard (aes), 2001.
<https://csrc.nist.gov/publications/detail/fips/197/final>
[dostęp: 02.03.2026].

- [12] National Institute of Standards and Technology. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac, 2007.
<https://csrc.nist.gov/publications/detail/sp/800-38d/final>
[dostęp: 02.03.2026].
- [13] Redgate Blog. Crow's foot notation.
<https://www.red-gate.com/blog/crow-s-foot-notation/>
[dostęp: 02.03.2026].
- [14] Michał Kozłowski. Wyniki testu procesu automatycznego oceniania sprawdzianów.
https://uniwroc-my.sharepoint.com/:w:/g/personal/339808_uwr_edu_pl/IQBMt8TsFnMgQbF49RD-TI0RAaChG0VAFr2fo5iXVyHlEiU?e=yu6Jkb
[dostęp 02.03.2026].